

Rapport de stage



Réalisé par MICHAUX-KINETAlexis

En préparation du diplôme du : Bachelor Universitaire de Technologie

Supervisé par : Christophe PRUD'HOMME GATEAU

Semestre 4 Année Universitaire 2024-2025



INTELLECTUAL PROPERTY - LIABILITY

This document written in the frame CSUM is the property of the University Montpellier (UM), legal representative of the University Space Centre of Montpellier (CSUM).

The authors of this document agrees to sell exclusively UM all economic rights they have on information that may be an intellectual property right. The rights assigned include, without limitation, the rights to use, reproduction, representation, adaptation, translation, distribution, operation, concession rights transferred, integration into another document, for all purposes to the UM communication, preparation or execution of any technical documents concerning the activities of the CSUM. This assignment will be for a period at least equal to the statutory term of the economic rights of an author in France, and this for all countries.

Reproduction of this document in whole or in part for the purposes of the activities of CSUM is permitted. Any dissemination of this document for purposes other than those of the CSUM is prohibited without prior written permission of the Director of CSUM.

Any information contained therein is the sole responsibility of the author. The Centre Spatial Universitaire is not responsible for any use that may be made of these information.



PROPRIETE INTELLECTUELLE - RESPONSABILITE

Ce document, rédigé selon la trame CSUM, est la propriété de l'Université Montpellier, représentant légal du Centre Spatial Universitaire de Montpellier (CSUM).

Les auteurs de ce document s'engagent à céder exclusivement à l'UM l'ensemble des droits patrimoniaux dont ils disposent sur les informations susceptibles de faire l'objet d'un droit de propriété intellectuelle. Les droits cédés comprendront notamment, les droits d'utilisation, de reproduction, de représentation, d'adaptation, de traduction, de distribution, d'exploitation, de concession des droits cédés, d'intégration dans un autre document, pour tous usages permettant à l'UM la communication, la préparation ou la réalisation de tout document technique concernant les activités du CSUM dans le cadre des projets ROBUSTA. Cette cession devra se faire pour une durée au moins égale à la durée légale des droits patrimoniaux d'un auteur en France, et cela pour tous pays.

Toute reproduction du présent document en tout ou partie pour les besoins des activités du CSUM est autorisée. Toute diffusion de ce document pour des besoins autres que ceux du CSUM est prohibée sauf autorisation préalable écrite du Directeur du CSUM.

Les informations contenues dans ce rapport n'engagent que son auteur. Le Centre Spatial Universitaire n'est pas responsable de l'usage qui pourrait en être fait.

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué au bon déroulement de mon stage et à l'enrichissement de mon expérience professionnelle.

Tout d'abord, je remercie chaleureusement **Christophe PRUD'HOMME GATEAU**, mon maître de stage, pour son accueil, sa bienveillance et le temps qu'il m'a consacré. Son encadrement et ses précieux conseils m'ont permis de mieux comprendre l'informatique dans le milieu professionnel.

Je remercie aussi le Professeur Laurent **DUSSEAU**, directeur du centre et le Docteur Muriel **BERNARD**, directrice lancements et réglementation, cheffe de la division pilotage.

Je remercie également la Fondation Van Allen pour le financement des 385 heures de mon stage au sein du CSUM.

Mes remerciements vont aussi à l'ensemble de l'équipe du Centre Spatial de l'Université de Montpellier pour leur disponibilité, leur patience et leur esprit de collaboration. Leur accompagnement m'a permis de m'intégrer facilement et d'évoluer dans un environnement de travail stimulant.

Enfin, j'exprime ma reconnaissance envers les enseignants de **IUT d'informatique de Montpellier** et plus particulièrement mon tuteur de stage M. HAETTEL pour son suivi et son soutien tout au long de cette période de stage.

Grâce à cette expérience, j'ai pu développer mes connaissances du milieu spatial professionnel, de l'administration système informatique et renforcer mon projet professionnel qui est de travailler dans le domaine du spatial. J'en garderai un souvenir enrichissant et d'autres opportunités se présentent au sein du CSUM.

Résumé

Ce document est un rapport de stage effectué au sein de Centre Spatial de l'Université de Montpellier et rédigé par un étudiant de l'IUT de Montpellier dans le cadre de sa deuxième année d'études. Il présente le travail effectué durant la période de stage, à la fois dans le développement logiciel et l'administration système et réseau.

Ce rapport parle des trois thématiques qui ont été abordées durant le stage :

1. Le développement et l'optimisation d'une solution de clonage et le déploiement de postes stagiaires via le réseau interne du CSUM. Cette partie traite de recherche d'une solution informatique, en utilisant le réseau RJ45, et aborde principalement le clonage et l'utilisation des serveurs.
2. Le développement d'un logiciel de traitement et de transfert de données issues d'alertes collisions vers un fichier excel. Cette partie traite de programmation en Python et l'utilisation d'une de ses librairies appelée Pandas. Il s'agissait de la conception d'un logiciel avec son optimisation.
3. Le développement et l'optimisation d'un outil utilisant Django, Cette partie abordera la schématisation et l'optimisation de l'outil appelé MCC*.

Mots-clés : MCC, Python, Django, Pandas, Excel, documentation, serveur, déploiement, réseau.

Sommaire

Introduction	10
1. Automatisation d'une solution pour le clonage de postes informatiques.	11
1.1. Cahier des Charges	11
1.1.1. Besoins fonctionnels	11
1.1.2. Besoins non fonctionnels	12
1.2. Rapport technique	12
1.2.1. Conception	12
1.2.2. Réalisation	14
1.1.2. Validation	17
1.1.3. Difficultés rencontrées	19
1.2. Apprentissages Critiques mobilisés	20
2. Automatisation de l'analyse des données issues des alertes collisions.	21
2.1. Cahier des Charges	22
2.1.1. Besoins fonctionnels	22
2.1.2. Besoins non fonctionnels	22
2.2. Rapport technique	23
2.2.1. Conception	23
2.2.2. Réalisation	29
2.2.3. Validation	39
2.2.4. Difficultés rencontrées	40
2.3. Apprentissages Critiques mobilisés	40
3. Refactorisation du code de la MCC	42
3.1. Cahier des Charges	42
3.1.1. Besoins fonctionnels	42
3.1.2. Besoins non fonctionnels	42
3.2. Rapport technique	42
3.2.1. Conception	42

Rapport de Stage au CSUM

3.2.2. Réalisation	43
3.2.3. Validation	45
3.2.4. Difficultés rencontrées	45
3.3. Apprentissages critiques mobilisés	46
4. Rapport d'activité pour l'ensemble du stage.	47
4.1. Installation de postes pour le personnel	47
5. Gestion de projet	48
Conclusion	49
Bilan du travail réalisé	49
Perspectives	49
Bilan des acquis techniques	49
Bilan de l'expérience en entreprise	50
Bilan de l'expérience personnelle	50
Bibliographie	51
Annexes	53

Table des figures

Figure 1 Schéma du clonage du postes stagiaire	11
Figure 2 Schéma création de l'image master	16
Figure 3 Schéma installation images depuis master	16
Figure 4 Modélisation installation terminé	17
Figure 5 Modélisation création alerte collision	21
Figure 6 Schéma réalisé par l'ESA sur l'évolution du nombre d'Object en orbite	22
Figure 7 Diagramme des cas d'utilisation Star Guardian	23
Figure 8 Modélisation objectif du projet	25
Figure 9 Modélisation architecture logiciel Star Guardian	25
Figure 10 Diagramme de class des Analyzer	26
Figure 11 Exemple de fichier txt de suivi de ROBUSTA-3A à la date du 2024-08-10	27
Figure 12 Architecture interface utilisateur	28
Figure 13 Logo de Star Guardian	29
Figure 14 Tableau de bord	30
Figure 15 Tri des fichiers	30
Figure 16 Fonction python organize_files_by_satellite()	30
Figure 17 Paramètres	33
Figure 18 Paramètres avec le thème blue	33
Figure 19 Génération du Excel	34
Figure 20 Exemple de fichier Excel de sortie du programme	35
Figure 21 Fonction python extract_value()	36
Figure 22 Fonction python extract_miss_distance()	36
Figure 23 Schéma conjonction	37
Figure 24 Fonction python is_conjonction()	37
Figure 25 Schéma du fonctionnement des pays	38
Figure 26 Fonction python get_operator_country_mapping()	39
Figure 27 Schéma utilisation de pyinstaller	39
Figure 28 Capture d'écran StarGuardian.exe	39
Figure 29 Schéma de la communication GS / Satellites	43
Figure 30 Schéma protocole encodeur / décodeur	43
Figure 31 Schéma héritage des codecs	44
Figure 32 Modélisation de la solution avec Routeur	44
Figure 33 Schéma detecteur de codec	45
Figure 34 Photo ordinateur démonté	47

Glossaire

CSUM : Centre Spatial de l'Université de Montpellier

Poste Master : Ordinateur préconfiguré servant de modèle pour cloner rapidement et efficacement les configurations et logiciels vers d'autres postes informatiques.

CubeSats : De petits satellites modulaires et compacts (une unité équivaut à un cube de 10 cm sur 10 cm sur 10 cm), conçus pour des missions spatiales à faible coût et à grande flexibilité.

CNES : Le Centre National d'Études Spatiales est l'agence spatiale française, responsable de la définition et de la mise en œuvre de la politique spatiale de la France.

ESA : European Space Agency est une organisation intergouvernementale européenne qui coordonne les efforts spatiaux des États membres pour développer des technologies et des missions spatiales.

UAR : Unité d'Appui à la Recherche

PXE : Le PXE permet à un ordinateur de démarrer depuis un réseau en chargeant un système d'exploitation ou un programme d'installation à partir d'un serveur, sans disque dur local.

MAC : Une adresse MAC (Media Access Control) est un identifiant unique attribué à chaque carte réseau. Elle permet d'identifier un appareil sur un réseau local.

SOLID : Les principes SOLID sont cinq règles de conception logicielle visant à rendre le code plus lisible, flexible et facile à maintenir.

LAN : Un LAN (Local Area Network) est un réseau informatique local qui connecte plusieurs appareils (ordinateurs, serveurs, imprimantes, etc.) sur une zone géographique restreinte, comme un bureau, une maison ou une école, permettant ainsi le partage de fichiers, de ressources et d'internet.

MCC : Le MCC est un programme interne au CSUM qui permet de contrôler et interagir avec les satellites en orbite.

ISO : Un fichier .iso est une image disque contenant une copie exacte d'un système de fichiers, souvent utilisée pour distribuer des logiciels ou des systèmes d'exploitation.

Bande S : La bande S est une plage de fréquences de 2 à 4 GHz utilisée principalement pour les communications par satellite, les radars et certaines technologies sans fil. Elle est appréciée pour sa capacité à traverser les obstacles comme la pluie et les nuages, ce qui la rend idéale pour des applications comme la télécommunication par satellite et la détection radar.

UHF : L'UHF (Ultra High Frequency) est une gamme de fréquences de 300 MHz à 3 GHz. Elle est utilisée pour les communications radio, la télévision, le Wi-Fi, et les systèmes de radar.

DSIN : La DSIN ou Direction du Système d'Information et du Numérique est l'organisme de l'université de Montpellier qui s'occupe de l'informatique à l'université.

NORAD : Le Commandement de la défense aérospatiale de l'Amérique du Nord est une organisation américano-canadienne dont la mission est la surveillance de l'espace aérien nord-américain.

Introduction

Ce rapport de stage présente le travail que j'ai réalisé au sein du Centre Spatial de l'Université de Montpellier (CSUM), dans le cadre de ma formation à l'IUT informatique de Montpellier.

Le Centre Spatial de l'Université de Montpellier est un centre dédié à la recherche, la conception et le développement de nanosatellites. Situé à Montpellier, il s'inscrit dans un cadre universitaire et se distingue par son rôle de formation, de recherche et d'innovation dans le domaine spatial. Son principal objectif est de former les étudiants par une approche pratique, en leur permettant de participer à des projets spatiaux réels, de la conception à l'exploitation des satellites.

Le CSUM est spécialisé dans les CubeSats*, des nanosatellites miniatures et standardisés, utilisés pour des missions scientifiques, éducatives ou technologiques, comme l'observation de la Terre ou la validation de nouvelles technologies.

Le centre joue également un rôle majeur dans la recherche et l'innovation, notamment en développant des technologies avancées telles que des systèmes embarqués et des logiciels de navigation. Il travaille en collaboration avec des agences spatiales comme le CNES, l'ESA, ainsi que des institutions académiques et des entreprises privées. Le CSUM est à l'origine de projets emblématiques, comme le ROBUSTA, l'un des premiers CubeSats français lancé le 13 février 2012, et contribue activement à l'écosystème spatial mondial à travers des partenariats et des initiatives collaboratives.

1. Automatisation d'une solution pour le clonage de postes informatiques.

Le CSUM accueille de nombreux stagiaires d'observation dans le cadre de sa mission de diffusion de la culture scientifique, afin de faire découvrir les métiers du spatial aux plus jeunes. Pour leurs travaux pratiques et leurs recherches durant leur semaine d'observation, des postes informatiques sont mis à leur disposition. Et à la fin de chaque session, ces postes sont entièrement réinitialisés pour accueillir les nouveaux stagiaires. Cependant, cette réinstallation, réalisée initialement à l'aide de l'outil Veeam Backup sur un disque USB, est un processus long et fastidieux, nécessitant un temps individuel considérable. Il s'agit de trouver le meilleur outil pour optimiser cette étape de réinitialisation en utilisant les possibilités du réseau informatique gigabit du CSUM.

1.1. Cahier des Charges

1.1.1. Besoins fonctionnels

- Depuis un poste informatique "Master", permettre le clonage des données et configurations vers plusieurs autres postes informatiques.
- Automatiser le plus possible la procédure de clonage, en minimisant l'intervention manuelle de l'utilisateur.
- Fournir une documentation détaillée et claire pour la procédure de clonage, afin que d'autres utilisateurs puissent suivre chaque étape sans confusion..

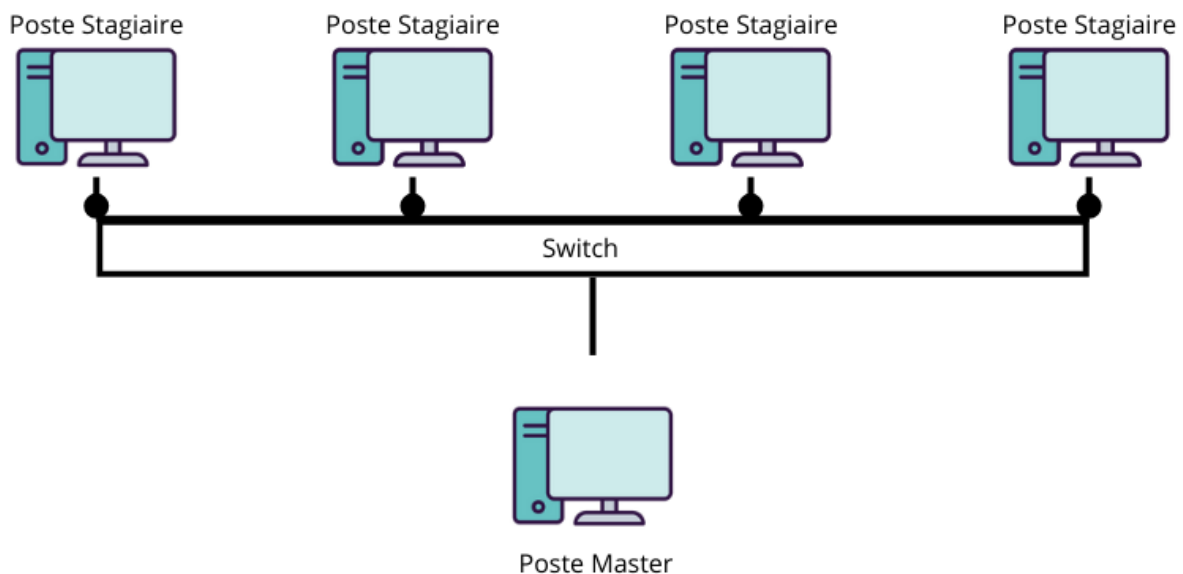


Figure 1 Schéma du clonage du postes stagiaire

1.1.2. Besoins non fonctionnels

- **Performance :**
 - Le processus de clonage des données et configurations entre les postes informatiques doit être rapide et efficace afin de minimiser le temps d'attente pour les utilisateurs.
- **Simplicité d'utilisation :**
 - L'interface permettant de cloner les données doit être simple et intuitive, avec une prise en main facile pour les utilisateurs, qu'ils soient expérimentés ou novices.
- **Fiabilité :**
 - Le processus de clonage doit être stable, sans risque de corruption des données ou de perte d'informations lors du transfert entre les postes.
- **Compatibilité :**
 - Le processus de clonage doit être compatible avec les différents types de matériels et systèmes d'exploitation utilisés au sein du CSUM, assurant une large portée d'application.
- **Gratuité.**

1.2. Rapport technique

1.2.1. Conception

1.2.1.1. Etudes des solutions

1.2.1.1.1. Veeam Backup

Veeam Backup est une solution de sauvegarde et de récupération de données conçue pour sécuriser les environnements virtuels, physiques et cloud. Elle permet aux entreprises de protéger efficacement leurs machines virtuelles, serveurs et applications grâce à des sauvegardes rapides et fiables. Grâce à des fonctionnalités avancées telles que la sauvegarde en temps réel, la réplication et la restauration, Veeam assure une protection optimale contre les pannes et la perte de données. Cependant, la version gratuite de test est limitée à une seule sauvegarde, et l'accès à l'ensemble des fonctionnalités requiert une version payante. De plus, cette version gratuite ne permet pas de démarrer des sauvegardes directement sur des machines via le réseau, obligeant ainsi l'utilisateur à créer un disque USB pour chaque machine.

1.1.1.1.1. Iventoy

IVentoy est une solution de gestion de sauvegarde et de récupération de données adaptée aux environnements IT modernes. Elle permet aux entreprises de protéger efficacement leurs systèmes, applications et fichiers. De plus, elle peut être utilisée pour démarrer un système via le réseau grâce au PXE*, à condition de disposer d'images ISO* des systèmes à cloner.

Dans cette optique, j'ai tenté de convertir une sauvegarde Veeam, composée de fichiers .vbm et .vbk, en une image .iso. Cependant, cette conversion s'est révélée inadaptée, car une fois l'ISO créée, la machine ne parvenait pas à démarrer, faute des données nécessaires. J'ai également tenté de générer une image ISO à partir d'une sauvegarde CloneZilla, mais le problème est resté identique.

1.1.1.1.2. CloneZilla Server

Clonezilla Server est une solution de clonage de disques et de déploiement de systèmes en masse, idéale pour les environnements nécessitant la gestion de plusieurs machines à la fois. Il permet de créer des images disque, de les déployer sur plusieurs ordinateurs simultanément et de réaliser des sauvegardes et restaurations efficaces. Clonezilla Server est particulièrement adapté aux administrations réseau et aux entreprises ayant besoin de déployer des configurations homogènes sur un grand nombre de machines.

1.1.1.1.3. Fog project

Fog Project est une solution open source permettant de gérer le déploiement et la virtualisation de machines dans des environnements distribués. Il facilite l'automatisation du provisionnement, de la configuration et de la gestion des systèmes d'exploitation sur des machines physiques ou virtuelles. Son interface web permet de gérer de manière centralisée ces processus, avec un support pour divers systèmes d'exploitation tels que Linux et Windows. Bien adapté aux entreprises et établissements éducatifs, Fog simplifie le déploiement à grande échelle, mais sa configuration initiale et son support peuvent être complexes.

Outils	Version	Points +	Points -
	6.3.0.177	Très simple d'utilisation, et très performant.	Dans sa version gratuite, il est impossible d'envoyer sur plusieurs machines en simultané à partir d'un réseau.
	1.0.19	Gratuit et facile d'utilisation.	Impossible de créer les images .iso non bootables nécessaire à son utilisation.
	2.5.1-16	Gratuit et open source.	Complicé à utiliser, basé sur un Debian et les commandes Linux associées.
	1.5.10.1639	Gratuit et open source	Manque de flexibilité, absence de support. Et impossibilité de clonage par le réseau du CSUM.

1.2.2. Réalisation

1.1.1.2. Création du poste Master

Tout d'abord il faut créer un poste master, basé sur un Windows 10 vierge en version .22.04. Et j'ai ensuite installé tous les logiciels demandés.

Type d'installation :

- **Général** : Applicable à tous les utilisateurs.
- **Personnel** : A configurer individuellement pour chaque session utilisateur.

Nom	Version	Utilité	Type d'installation
F-secure UM	16.0	Anti-virus de l'université de Montpellier	Général
Chrome	124	Navigateur internet	Général
FireFox	134	Navigateur internet	Général
VeamBackup	6.3.0.177	Agent de sauvegarde	Général
7zip	2409	Décompresseur de fichier .zip	Général
Office	2013	Suite office 2013	Général
Vlc	3.0.21	Lecteur vidéo	Général
OpenVPN	3.6.0.4047	Vpn	Personnel
AcrobateReader	24.003	Editeur de PDF	Général
Ptest	1.33.075	Outil de test de performance	Général
Notepad++		Éditeur de texte	Général
Citadel	8.4.1	Outils de communication sécurisée	Personnel
Keepass	2.57.1	Gestionnaire de mots de passe	Personnel
NextCloud	3.15.3	Serveur de fichiers	Personnel
SolidWorks	2022R3	Outil de modélisation 2D / 3D	Personnel

1.2.2.1. Choix de la solution

Dans un premier temps, j'ai utilisé **Veeam Backup** pour tester la solution qui était utilisée initialement. Cependant, elle présentait une limite importante : elle ne permettait de cloner qu'une machine à la fois. De plus, le processus est relativement long, nécessitant environ **40 minutes** pour transférer une image de **187 Go** sur une machine stagiaire.

J'ai ensuite testé **IVentoy**, qui offrait la possibilité de transférer des images via le réseau. Toutefois, cet outil ne supportait que les fichiers au format **.iso**. J'ai tenté de convertir l'image extraite de Veeam Backup (**.vbk**) en **.iso** en utilisant **WinCDEmu (4.1)**, mais l'image générée n'était pas bootable, rendant IVentoy inutilisable dans ce contexte.

Face à ces contraintes, je me suis tourné vers **CloneZilla Serveur**, qui répondait parfaitement à mon besoin grâce à sa capacité à cloner un disque dur et à transférer les images via le réseau. Après plusieurs tests, j'ai constaté que l'outil fonctionnait bien, mais ne permettait pas de cloner une machine à une autre directement via le réseau interne du CSUM. J'ai alors mis en place un réseau local en utilisant un **switch** autonome (LAN) pour contourner cette limitation, et cette fois-ci, CloneZilla a fonctionné efficacement.

Enfin, j'ai envisagé une dernière alternative : **FOG Project**, une solution open-source avec une interface graphique claire et intuitive. Bien que très simple d'utilisation, cette solution ne fonctionnait pas car la machine ciblée n'arrivait pas à atteindre le serveur FOG.

Ainsi, après ces différents essais, **CloneZilla Serveur s'est imposé comme la solution la plus efficace et adaptée**, malgré une prise en main initiale un peu complexe. Son principal avantage réside dans sa capacité à cloner et déployer des machines via le réseau, ce qui répond parfaitement à la problématique du stage.

1.1.1.3. Utilisation de DRBL (CloneZilla Serveur)

Vous pouvez trouver toute la démarche à suivre dans le rapport technique sur l'utilisation en annexe.

1.1.1.3.1. Etape 1 : Création de l'image

Il faut lancer CloneZilla sur la machine master afin de copier son disque dur (Il faut la booter depuis une clé). Cette machine doit posséder tous les logiciels / informations que nous voulons cloner sur les autres. Ainsi CloneZilla va créer une copie compressée de la machine stagiaire dans un dossier qu'il peut lire et ensuite je peux l'exporter sur un disque dur externe USB, pour être facilement déplaçable d'un poste cible à un autre.

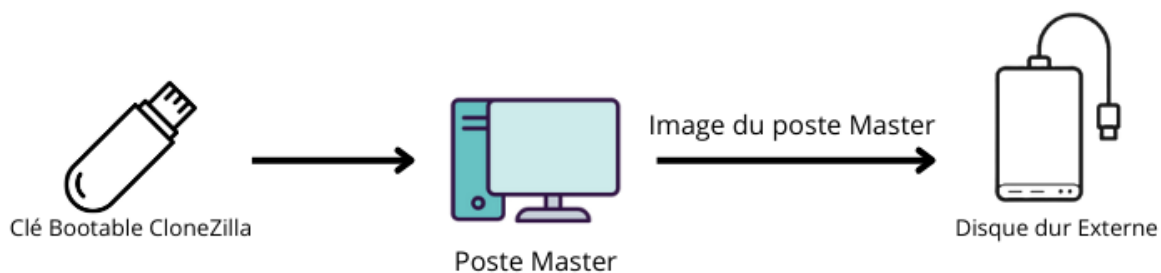


Figure 2 Schéma création de l'image master

1.1.1.3.2. Etape 2 : Transfert de l'image

Une fois que j'ai l'image du Master je peux la placer sur un serveur et la monter sur le disque. De là je peux lancer le serveur DRBL (cloneZillaServer) et démarrer les machines stagiaires cible qui vont grâce à la technologie PXE IPv4* booter sur le serveur DRBL. Ainsi l'ordinateur stagiaire va pouvoir récupérer les données se trouvant sur le serveur et les récupérer afin de s'installer sur le disque.

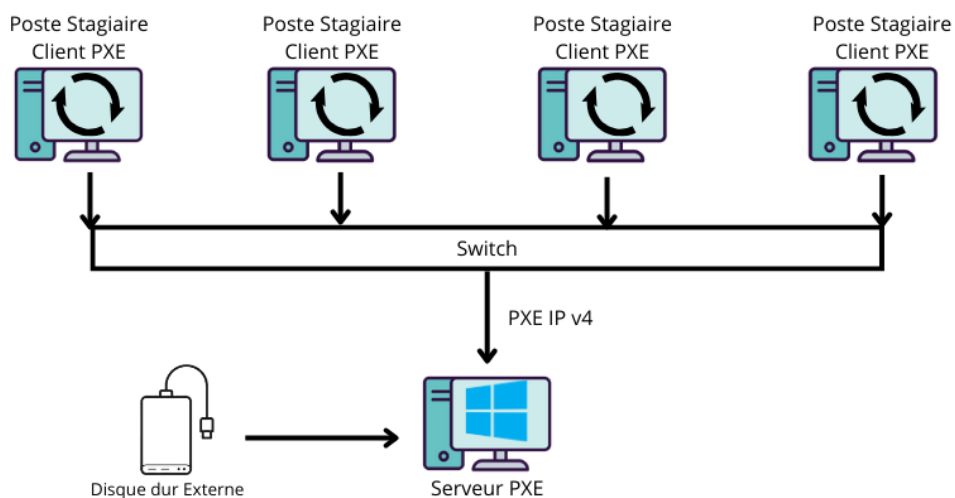


Figure 3 Schéma installation images depuis master

Nous nous retrouvons alors avec plusieurs postes ayant les même données et logiciels que le poste Master, ce sont des clones de celui-ci.

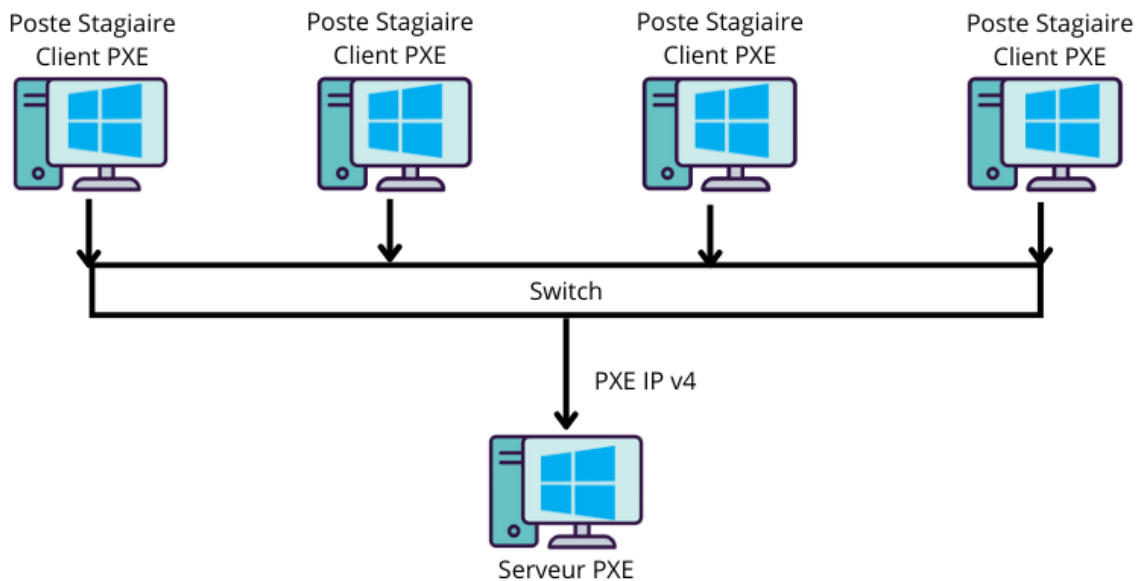


Figure 4 Modélisation installation terminé

1.1.1.3.3. Etape 3 : Personnalisation des comptes

Une fois les postes installés il faut aller dans les paramètres et modifier le nom des postes suivant la logique demandée et d'attribuer une IP fixe associée à l'adresse MAC de la carte réseau du poste stagiaire.

1.1.2. Validation

1.1.2.1. Test / Optimisation

Une fois la technologie choisie j'ai effectué de nombreux tests pour optimiser le transfert.

J'ai tout d'abord essayé avec une image windows vide de 30 go et j'ai testé pour voir les différentes manières d'utiliser CloneZilla server et le clonage de postes.

Caractéristiques de la machine master :

OS	Windows 10 22.04
RAM	32 go (2 x 16 go) de mémoire DDR3
Stockage	Disque 1T ssd
Processeur	Intel Xeon E5-2609 v4 @ 1.79GHz bi processeur
Carte graphique	Nvidia Geforce GTX 1080

Rapport de Stage au CSUM

J'ai d'abord essayé d'utiliser la fonctionnalité de clonage propre à CloneZilla server mais ce processus était très long.

Nombre de machines clientes	Temps
1	45 min
2	60 min
3	60 min

En effet la création de l'image étant réalisée au fur et à mesure de l'envoi le poste master ne pouvait pas facilement envoyer les données. Aussi cela posait un autre problème, car l'objectif était de placer CloneZilla server sur un Serveur dédié (Ici une Machine physique) et ainsi il aurait fallu que le serveur accueille l'environnement Windows complet pour être utilisé. Aussi si nous voulions héberger plusieurs images il aurait fallu héberger les partitions des autres postes ce qui aurait été très coûteux en ressources et en capacité de calcul pour un résultat médiocre.

C'est ainsi que j'ai utilisé la deuxième solution de CloneZilla server qui utilise une image CloneZilla d'un poste que nous pouvons créer sur n'importe quelle machine et l'exporter sur un disque dur externe transportable. On peut ainsi facilement transporter plusieurs images sur un même disque. Et aussi lors du transfert, le temps était moindre.

Nombre de machines clientes	Temps
1	5 min
2	8 min
3	11 min
4	30 min

Néanmoins je pouvais faire toujours plus rapide, en transférant les images directement sur le disque de la machine. Cette procédure a pour effet de réduire le nombre de périphériques et aussi de stocker directement les images sur un serveur. Et aussi puisque la machine travaille directement sur son disque, elle n'a pas besoin d'aller les chercher sur un périphérique externe qui est limité par sa vitesse de transfert et son câble ainsi que son port USB-A ou USB-C.

Pour finir les tests j'ai effectué le transfert de l'image de 187 go (l'image du Master), en utilisant toutes les méthodes tester précédemment. Je suis ainsi arrivé à ce résultat :

Nombre de machines clientes	Temps
1	52 min
2	55 min
3	1 h 20

1.1.2.1. Conclusion

Avec les tests effectués il a été montré qu'en fonction de la demande l'outil privilégié est différent. En effet s'il s'agit d'une simple installation sur un postes uniques l'utilisation de Veeam reste toujours plus facile à mettre en place néanmoins à partir de 2 postes la solution de CloneZilla Server devient intéressante car pour le même temps on peut faire 2 installations.

1.1.3. Difficultés rencontrées

1.1.3.1.1. Problème de /home/partimag

Lors du lancement, CloneZilla server récupère les données des images dans le dossier /home/partimag néanmoins il y a eu un problème de transfert de données du disque dur externe provoquant l'arrêt du programme.

Je l'ai résolu en copiant directement l'image sur le disque interne et en la plaçant dans /home/partimag ce qui a permis une accélération de 5 à 10 min en moyenne car cloneZilla n'avait plus besoin de récupérer les données sur un disque dur externe.

1.1.3.1.2. Problème de transfert par le réseau

Il y a eu un problème avec le transfert par réseau de l'UM. En effet nous n'avions pas accès à l'IP du serveur PXE telle qu'elle était configurée dans le switches. Il a ainsi fallu demander à la DSIN* une copie de la configuration des switches demandée par l'ancien admin système du CSUM. Une fois récupéré j'ai pu connaître l'IP que je pouvais attribuer au serveur PXE ce qui a permis le fonctionnement par réseau de l'UM.

1.1.3.1.3. Problème pour passer par Iventoy

J'ai eu beaucoup de mal car Iventoy était la solution prévue à la base j'ai ainsi essayé par de nombreux moyens de transformer les données backup de Veeam Backup / cloneZilla vers un fichier

.iso mais le problème étant qu'elle était non bootable, elle était ainsi non utilisable par les machines cible.

1.2. Apprentissages Critiques mobilisés

- **AC23.02** : Utiliser des serveurs et des services réseaux virtualisés
- **AC33.02** : Configurer un serveur et des services réseaux de manière avancée (virtualisation...)
- **AC13.01** : Identifier les différents composants (matériels et logiciels) d'un système numérique
- **AC13.03** : Installer et configurer un système d'exploitation et des outils de développement

2. Automatisation de l'analyse des données issues des alertes collisions.

Le CSUM exploite des satellites en orbite autour de la Terre. Avec l'augmentation du nombre de satellites en orbite basse, le risque de collision s'accroît considérablement. Pour prévenir ces incidents, le NORAD* envoie des alertes aux propriétaires des satellites. Toutefois, ces notifications sont nombreuses et de plus en plus fréquentes, rendant leur traitement long et fastidieux. Afin d'optimiser ce processus, le docteur BERNARD m'a demandé de concevoir un algorithme permettant de transférer automatiquement les données des fichiers vers un fichier Excel et de les analyser, lui faisant ainsi gagner un temps précieux.

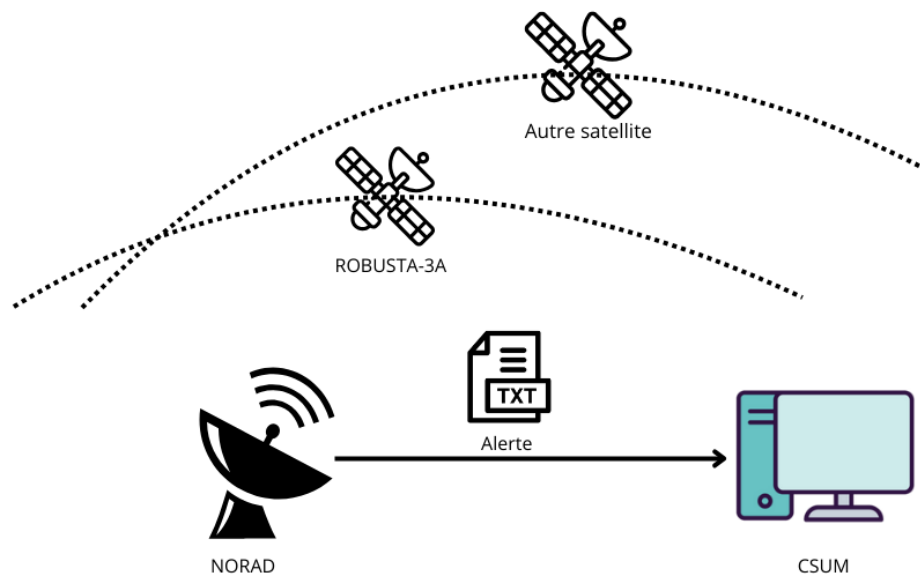


Figure 5 Modélisation création alerte collision

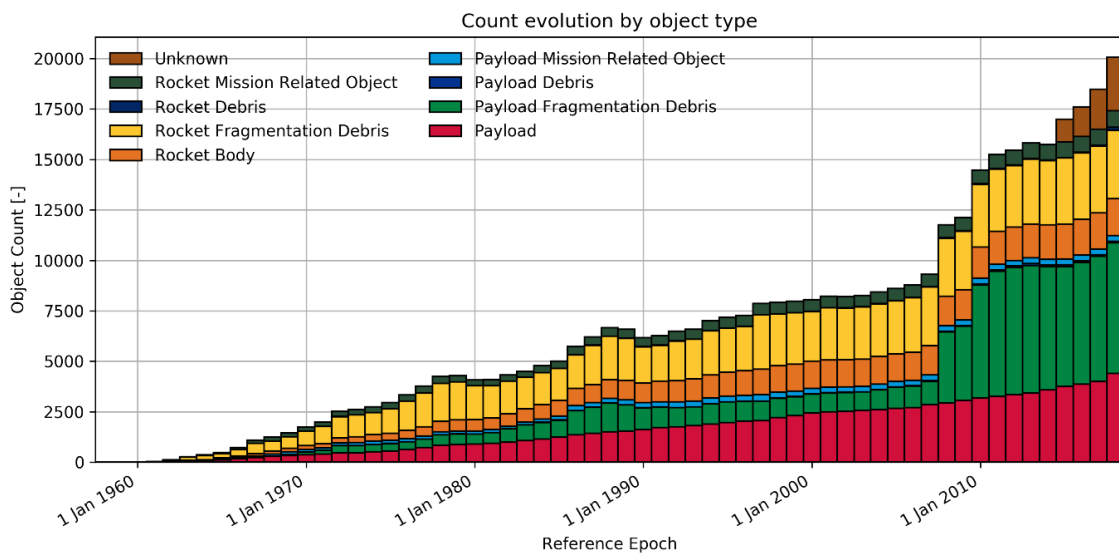


Figure 6 Schéma réalisé par l'ESA sur l'évolution du nombre d'Object en orbite

2.1. Cahier des Charges

2.1.1. Besoins fonctionnels

- Importer et analyser les données contenues dans des fichiers .txt générés avec chaque alerte de collision, puis les centraliser dans un fichier Excel tout en assurant leur traitement de manière automatique.
- Générer automatiquement des graphiques à partir de ces mêmes fichiers .txt et excel.

2.1.2. Besoins non fonctionnels

- Performance :
 - L'application doit pouvoir traiter un grand volume de fichiers (3 à 4 par jour par satellite en vol) rapidement, sans ralentissements notables.
 - Les opérations d'analyse et de génération de graphiques doivent être optimisées pour éviter une surcharge des ressources système.
- Fiabilité :
 - Les données doivent être fiables pour garantir leur exploitation ultérieure.
- Simplicité d'utilisation :
 - L'interface utilisateur doit être intuitive et accessible, même pour des utilisateurs ayant peu de compétences techniques.
 - Les processus, comme l'importation des fichiers ou la génération des graphiques, doivent être largement automatisés pour minimiser les interactions manuelles.
 - Le logiciel doit être disponible au format .exe afin de simplifier son exécution sous Windows.

2.2. Rapport technique

2.2.1. Conception

2.2.1.1. Diagramme des cas d'utilisation

Le diagramme illustre les interactions entre l'acteur utilisateurs et les fonctionnalités principales du système.

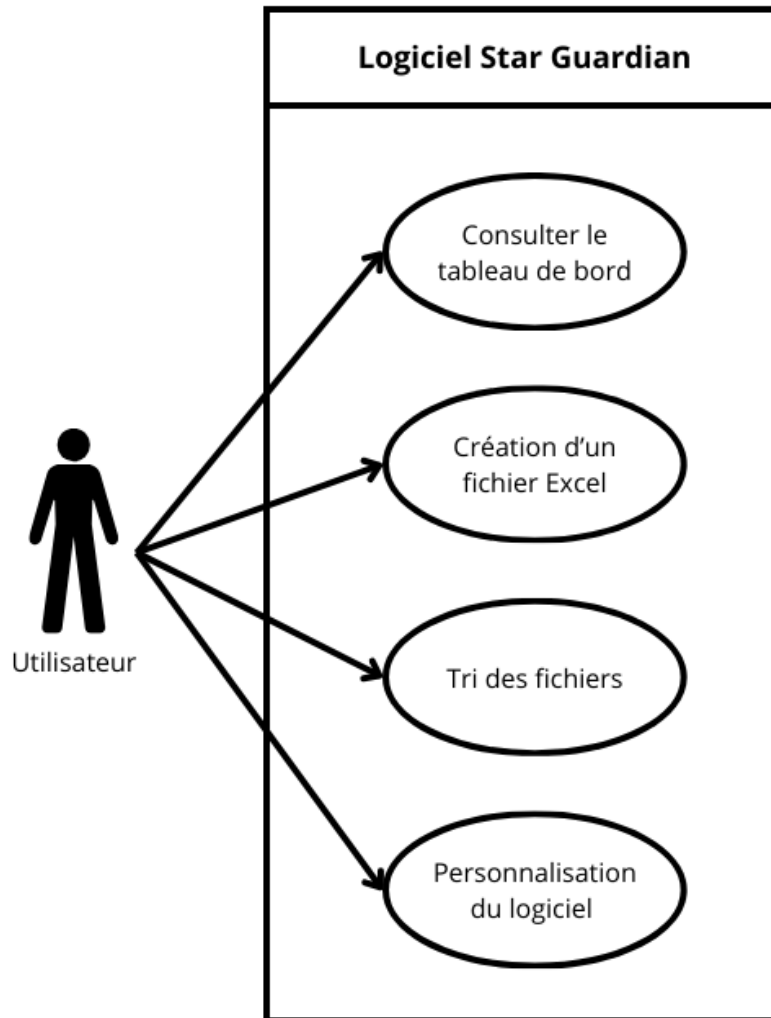


Figure 7 Diagramme des cas d'utilisation Star Guardian

Pour compléter le diagramme, chaque cas d'utilisation est présenté avec une description succincte. Ces descriptions couvrent :

Scénario nominal : déroulement standard et attendu de l'interaction.

Autres scénarios : variations possibles, erreurs rencontrées, ou comportements alternatifs.

2.2.1.2. Cas d'utilisation

Consulter le tableau de bord

Scénario nominal : Après avoir effectué une analyse, l'utilisateur peut consulter les principales données, telles que :

- Le nombre de fichiers traités
- Le nom du satellite
- Le nombre de conjonctions détectées
- Les dates d'étude

Création d'un fichier Excel

Scénario nominal : L'utilisateur sélectionne un dossier contenant des fichiers d'alertes de collision au format .txt. Une fois l'analyse effectuée, le logiciel propose un emplacement pour enregistrer le fichier Excel généré.

Trie des fichiers

Scénario nominal : L'utilisateur dispose d'un dossier contenant des fichiers d'alertes de collision relatifs à plusieurs satellites. Après traitement, le logiciel génère automatiquement un dossier distinct pour chaque satellite, regroupant les fichiers correspondants.

Personnalisation du logiciel

Scénario nominal : L'utilisateur peut personnaliser l'interface du logiciel, notamment en modifiant le thème (par exemple, passer à un thème bleu).

Autres scénarios :

- Définir les répertoires par défaut des fichiers d'entrée (emplacement des fichiers sources)
- Définir le dossier de sortie par défaut
- Configurer le fichier de paramètres utilisé par le logiciel

2.2.1.3. Architecture générale

L'application s'appuie sur une architecture simple développée en python qui s'exécute en local.

Le principe général réside dans l'ouverture d'un dossier et des fichiers qu'il contient. On parcourt les données contenues dans chaque fichier .txt. Une fois fait les données vont être traitées par des scripts qui vont les exporter dans un fichier excel, ou dans des graphiques.

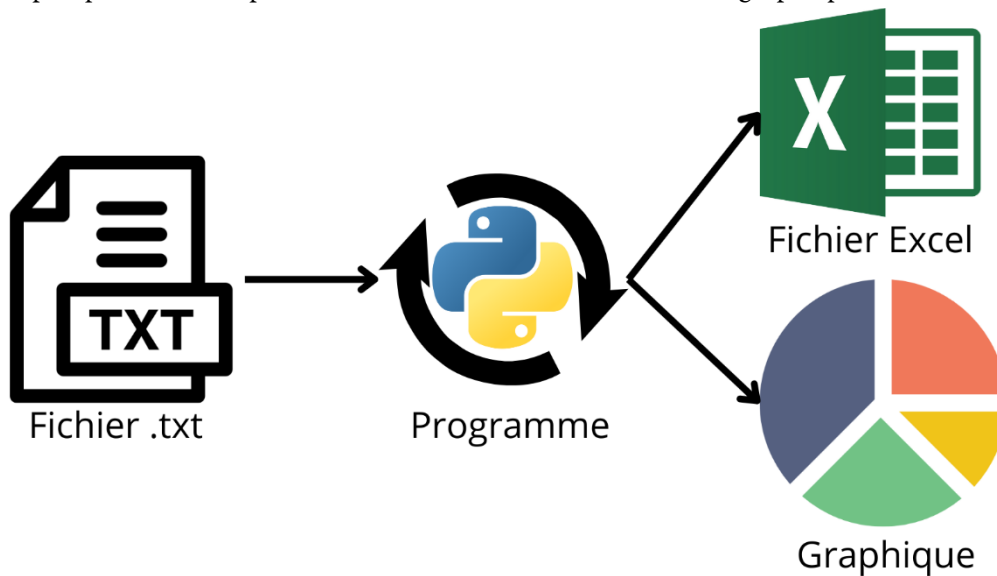


Figure 8 Modélisation objectif du projet

Le projet possède plusieurs modules, un pour chaque fonctionnalité. Le premier est l'interface graphique qui gère toute l'UI du logiciel, celle-ci est codée en Tkinter.

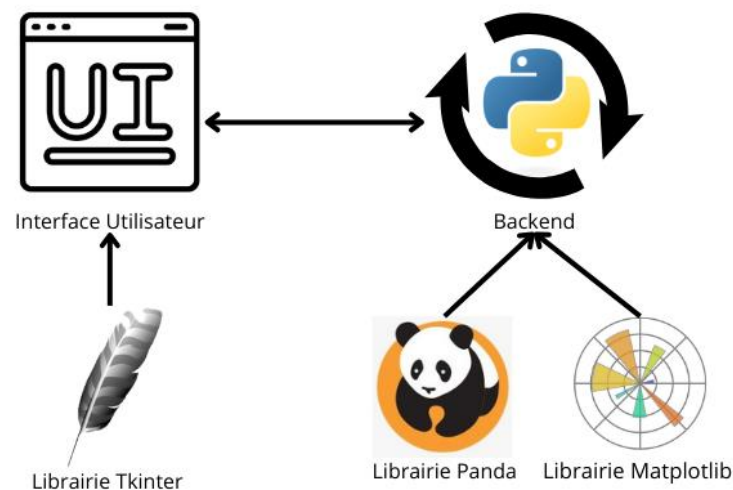


Figure 9 Modélisation architecture logiciel Star Guardian

Ensuite le gestionnaire de la création des graphiques. Puis les scripts d'extraction qui vont récupérer les données avant de les traiter et de les insérer dans l'excel.

Dans ce but une architecture modulaire a été privilégié, où des modules ont été conçus selon une structure héritée. La classe **BaseAnalyzer** abstraite possède tous les attributs communs, tels que les chemins d'entrée (le dossier contenant les données) et les chemins de sortie (le fichier Excel où les données seront stockées). Chaque type de donnée ou analyse spécifique peut alors hériter de la classe **BaseAnalyzer**, ce qui permet d'ajouter facilement de nouvelles analyses de données. (le logiciel est basé sur les principes SOLID*).

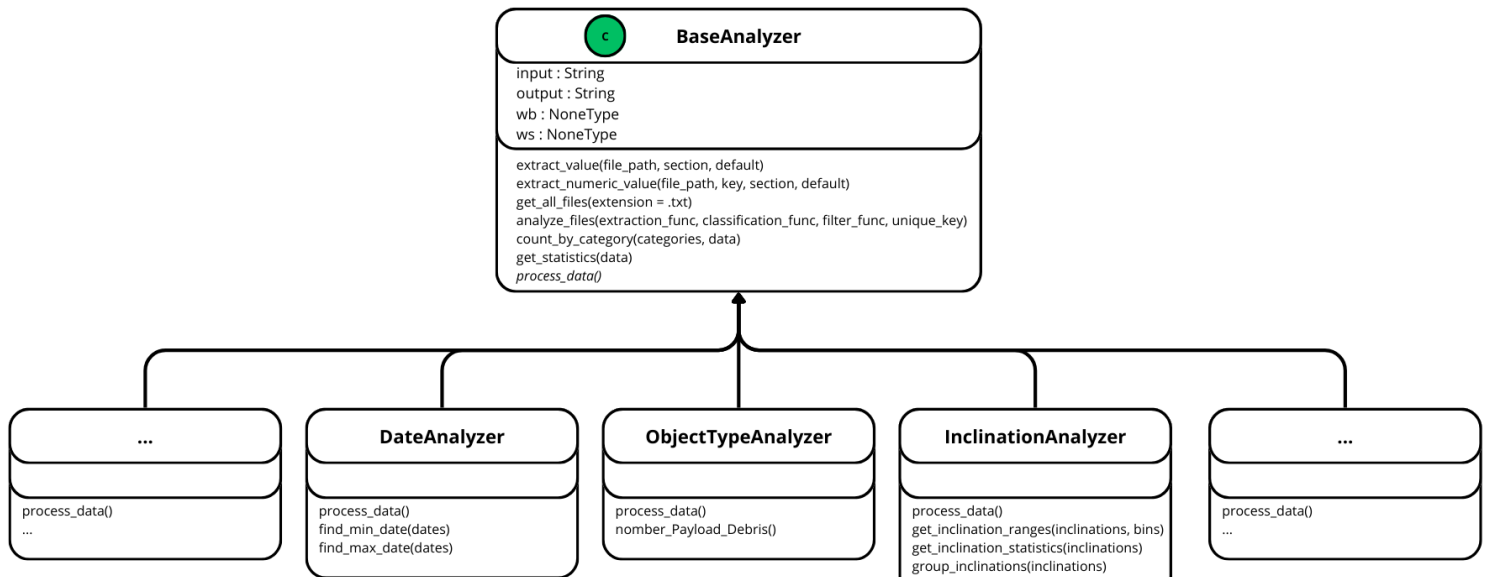


Figure 10 Diagramme de class des Analyzer

Cette classe possède aussi une méthode permettant de parcourir le dossier et d'ouvrir les fichiers et de rechercher les données demandées.

Les fichiers suivent une structure spécifique, ce qui me permet de les lire ligne par ligne et d'extraire les données dès la détection d'un signe « = ».

```

CCSDS_CDM_VERS      =1.0
COMMENT              =CDM_ID:788084516
CREATION_DATE        =2024-08-10T12:21:06.000000
ORIGINATOR           =CSpOC
MESSAGE_FOR          =ROBUSTA-3A
MESSAGE_ID           =000060243_conj_000000115_2024226121951_22312170182598
TCA                  =2024-08-13T12:19:51.661000
MISS_DISTANCE        =351           [m]
RELATIVE_SPEED       =10351         [m/s]
RELATIVE_POSITION_R  =-80           [m]
RELATIVE_POSITION_T  =248.8         [m]
RELATIVE_POSITION_N  =234.7         [m]
RELATIVE_VELOCITY_R  =-28.5         [m/s]
RELATIVE_VELOCITY_T  =-7072.5       [m/s]
RELATIVE_VELOCITY_N  =7559.2        [m/s]
COLLISION_PROBABILITY =0.000002604368
COLLISION_PROBABILITY_METHOD =FOSTER-1992
  
```

Figure 11 Exemple de fichier txt de suivi de ROBUSTA-3A à la date du 2024-08-10

Cependant, un problème persiste : ces fichiers contiennent des informations pour deux satellites (celui appartenant au CSUM et celui de l'alerte). Un tiers des données est commun aux deux (comme la vitesse relative, etc), tandis que le reste est spécifique à chaque satellite. Étant donné que les données du satellite appartenant au CSUM sont peu pertinentes, il est crucial de ne traiter que celles du satellite 2.

2.2.1.4. Technologies utilisées

2.2.1.4.1. Python

Python a été choisi pour sa simplicité, sa flexibilité et la richesse de son écosystème. Ce langage permet une manipulation efficace des fichiers et des données, tout en offrant des bibliothèques puissantes pour le traitement et la visualisation des informations. De plus, Python facilite l'automatisation des tâches grâce à ses nombreuses fonctionnalités dédiées à la gestion de fichiers et au traitement des données.

2.2.1.4.2. Pandas

Pandas est une bibliothèque essentielle pour la manipulation et l'analyse de données en Python. Elle a été utilisée pour :

- Lire et extraire les données contenues dans les fichiers .txt générés lors des alertes de collision.
- Nettoyer et structurer ces données en DataFrames, un format optimisé pour l'analyse.

- Centraliser les informations et les exporter vers un fichier Excel pour faciliter leur exploitation.

Grâce à Pandas, l'importation, la transformation et l'analyse des données ont pu être automatisées de manière efficace et rapide.

1.2.1.1.1. Matplotlib

Matplotlib est une bibliothèque de visualisation en Python permettant de générer divers types de graphiques.

1.2.1.1.2. Tkinter

Tkinter est une bibliothèque graphique pour Python qui permet de créer des interfaces utilisateur. Tkinter permet de concevoir des fenêtres, des boutons, des champs de texte, des menus, et bien plus, tout en utilisant une syntaxe claire et concise. C'est un excellent choix pour des applications légères.

2.2.1.5. Interface utilisateur

L'interface utilisateur est composée de cinq blocs :

- Le tableau de bord, qui sert à afficher les données.
- Le bloc de génération des données vers Excel, chargé de l'affichage des commandes permettant le transfert des données vers Excel.
- Le bloc de tri des fichiers, qui permet de demander et de lancer le tri des fichiers.
- Le bloc de génération de graphiques, qui permet de créer les graphiques.
- Le bloc des paramètres et de la personnalisation.

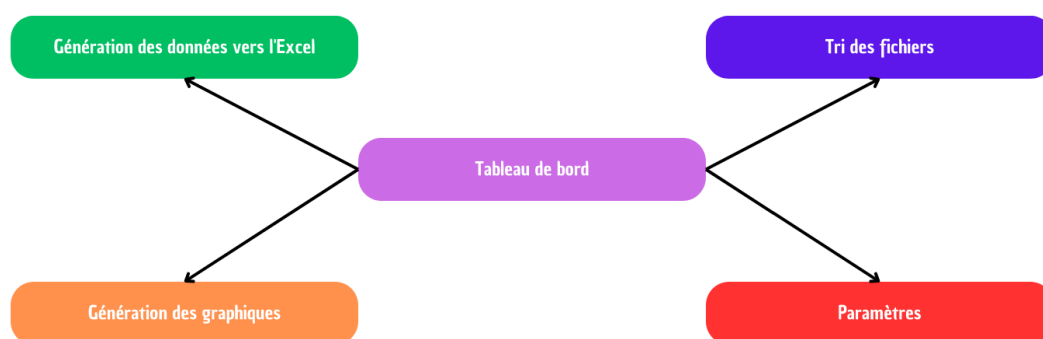


Figure 12 Architecture interface utilisateur

2.2.2. Réalisation

2.2.2.1. Présentation générale

Voici l'application **Star Guardian**, dont j'ai eu la liberté de choisir le nom. Ce programme dispose d'une interface graphique développée avec **Tkinter**, une bibliothèque Python facilitant la création d'interfaces visuelles tout en assurant une connexion fluide avec le backend. (*Vous trouverez le guide d'utilisation en annexe.*) L'objectif principal de cette interface graphique est de rendre l'application **accessible et intuitive**, en particulier pour les utilisateurs sans formation en informatique ou en programmation. Une interface en ligne de commande peut rapidement devenir complexe et difficile à utiliser pour un non-initié ce qui est le cas pour le docteur BERNARD. Grâce à cette approche visuelle, l'utilisateur peut interagir avec le programme de manière plus naturelle, sans avoir à manipuler directement du code.



Figure 13 Logo de Star Guardian

2.2.2.2. Tableau de Bord

Pour afficher les données principales j'ai réalisé une interface graphique. Ici sont affichés le nombre de fichiers traités, le nombre de conjonctions, les dates utilisées pour la période et le nom du satellite.



Figure 14 Tableau de bord

2.2.2.3. Tri des fichiers

Pour pouvoir utiliser efficacement les données il faut qu'elles soient triées par satellites pour que les données soient utilisables et intéressantes. Ainsi j'ai écrit un script récupérant le nom du satellite sur chacun des fichiers .txt et crée un dossier pour chaque satellite, facilitant ainsi l'exploitation.

The 'Tri des fichiers' interface includes two file selection sections. The first section, 'Répertoire source:', has an input field and a 'Parcourir' button. The second section, 'Répertoire cible:', also has an input field and a 'Parcourir' button. A 'Trier les fichiers' button is located at the bottom center of the form.

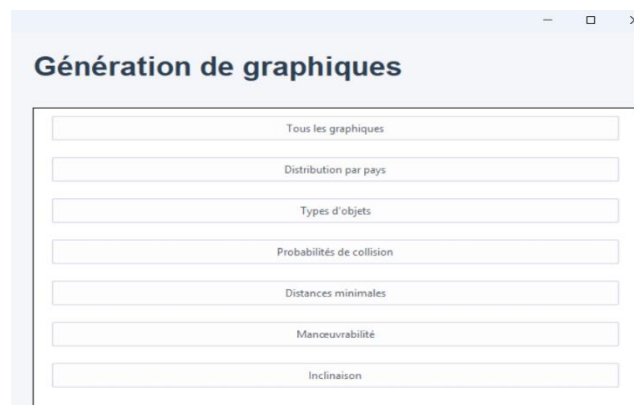
Figure 15 Tri des fichiers

```
def organize_files_by_satellite(source_dir, target_dir):  
    """ Organise les fichiers dans des dossiers basés sur le premier OBJECT_NAME  
    trouvé et les place dans le dossier cible.  
    Args:  
        source_dir (str): Répertoire contenant les fichiers à organiser.  
        target_dir (str): Répertoire où les fichiers triés seront placés."""  
    if not os.path.exists(source_dir):  
        print(f"Le répertoire {source_dir} n'existe pas.")
```

Figure 16 Fonction python `organize_files_by_satellite()`

```
        return
    # Parcours des fichiers dans le répertoire source
    for filename in os.listdir(source_dir):
        filepath = os.path.join(source_dir, filename)
        # Vérifie si c'est un fichier
        if os.path.isfile(filepath):
            try:
                # Lecture du contenu du fichier
                with open(filepath, 'r', encoding='utf-8') as file:
                    content = file.read()
                # Recherche du premier OBJECT_NAME
                match = re.search(r'OBJECT_NAME\s*=\s*([^\s]+)', content)
                if match:
                    satellite_name = match.group(1)
                    # Création du dossier correspondant dans le répertoire cible
                    satellite_dir = os.path.join(target_dir, satellite_name)
                    os.makedirs(satellite_dir, exist_ok=True)
                    # Déplacement du fichier
                    shutil.move(filepath, os.path.join(satellite_dir, filename))
                    print(f"Fichier {filename} déplacé vers {satellite_dir}")
                else:
                    print(f"OBJECT_NAME introuvable dans {filename}.")
            except Exception as e:
                print(f"Erreur lors du traitement du fichier {filename}: {e}")
```

2.2.2.4. Génération des graphiques



```
def generer_graphique_avec_legend(self, data, title, categories_order):
    # Prépare les données pour le graphique
    labels = []
    sizes = []
    for category in categories_order:
        count = data.count(category)
```

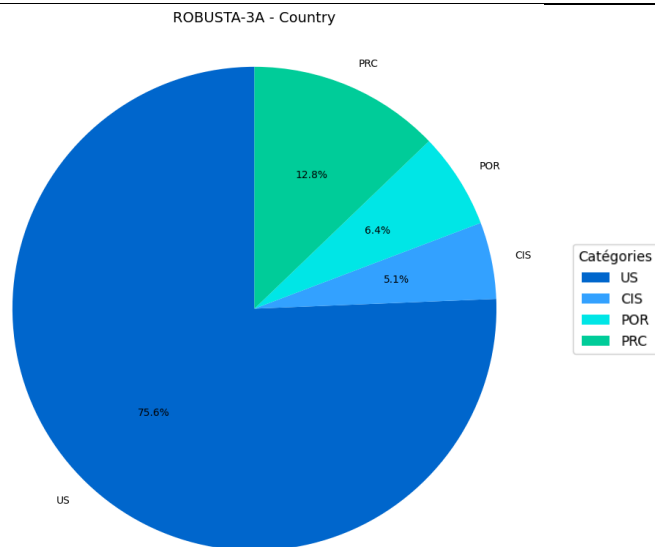
```

    if count > 0: # N'inclut que les catégories non vides
        labels.append(category)
        sizes.append(count)

    # Création du graphique en camembert
    plt.figure(figsize=(12, 8))
    patches, texts, autotexts = plt.pie(sizes, labels=labels,
    colors=self.colors[:len(labels)], autopct='%1.1f%%', startangle=90)

    plt.axis('equal') # Égalise l'aspect du camembert
    plt.title(title, fontsize=14, pad=20)
    plt.legend(patches, labels, # Ajout de la légende
               title="Catégories",
               loc="center left",
               bbox_to_anchor=(1.2, 0.5))
    plt.tight_layout()
    plt.savefig(chemin_complet, bbox_inches='tight', facecolor='white')
    plt.close()

```



2.2.2.5. Paramètres

La section Paramètres permet à l'utilisateur de personnaliser l'application afin d'améliorer son efficacité. Il peut notamment définir des répertoires par défaut, comme le répertoire sources, pour faciliter l'importation des données.

Paramètres

Configuration de l'application

Chemins par défaut

Répertoire source par défaut: Parcourir

Répertoire cible par défaut: Parcourir

Modèle Excel: Parcourir

Format du modèle: ☒ Microsoft Excel (*.xlsx)

Interface utilisateur

☒ Mettre à jour automatiquement le tableau de bord après l'analyse

Thème: Appliquer

Figure 17 Paramètres

L'utilisateur a également la possibilité de sélectionner le format du fichier de sortie, entre LibreOffice ou Excel.

Et pour finir, la section « Interface utilisateur » qui permet de modifier le thème (Annexe ..) de l'application pour adapter son apparence selon ses préférences.

Figure 18 Paramètres avec le thème blue

2.2.2.6. Génération des données vers l'Excel

2.2.2.6.1. Présentation générale

La génération des données vers les documents classeur repose sur une architecture simplifiée. Des fichiers modèles prédéfinis sont utilisés, comportant une mise en page avec des couleurs, des graphiques vides et des champs réservés. Le logiciel crée une copie de ces modèles et l'ouvre en mode écriture. Simultanément, il parcourt les fichiers de données au format **.txt** présents dans le dossier source, puis insère les données extraites dans la copie du modèle.

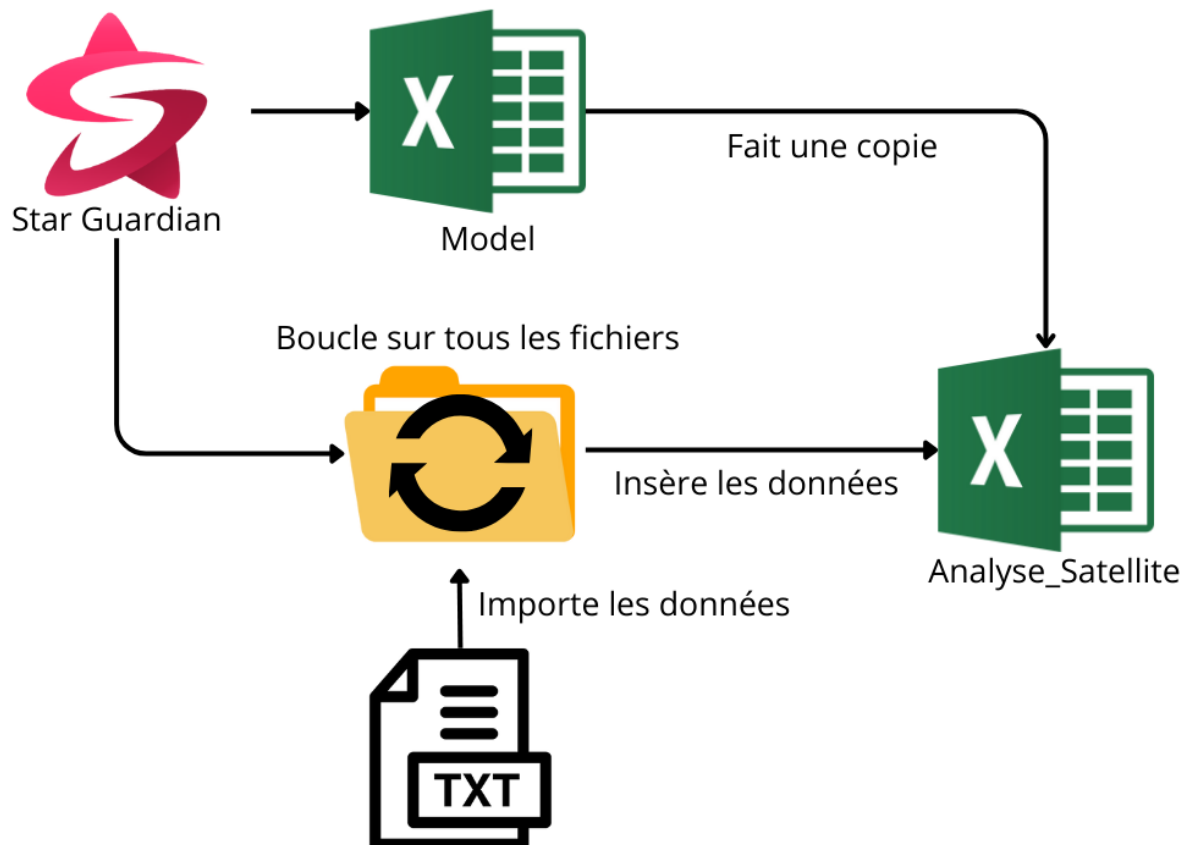


Figure 19 Génération du Excel

2.2.2.6.2. Fichier Excel

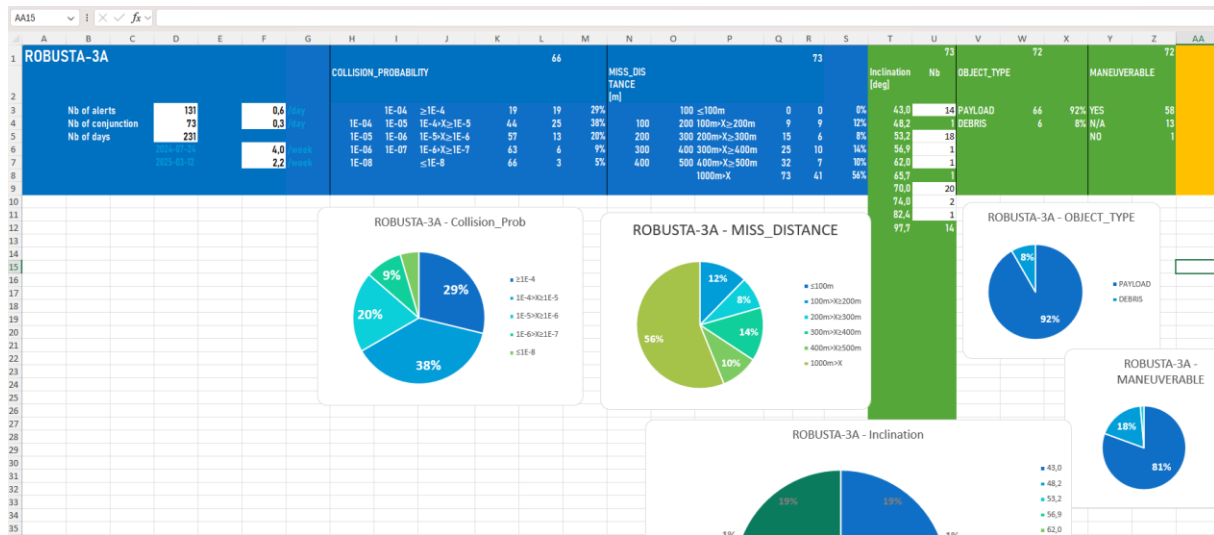


Figure 20 Exemple de fichier Excel de sortie du programme

2.2.2.6.3. Données classiques

Ainsi j'ai créé le paramètre section qui va parcourir le fichier jusqu'à arriver sur le titre de la section (OBJECT 1 ou OBJECT 2) et une fois fait va étudier les données présentes seulement dans cette section. Pour optimiser la recherche, j'ai donc parcouru l'ensemble du fichier jusqu'à identifier la section correspondant au satellite 2. Ensuite, j'ai structuré l'algorithme de manière à ne traiter que cette partie du fichier, réduisant ainsi le volume de données analysées à un tiers du total. En somme, j'ai partitionné le fichier afin de maximiser l'efficacité du traitement.

J'ai conçu un algorithme qui ouvre chaque fichier .txt et, en le parcourant, extrait les informations demandées.

```
def extract_value(self, file_path, key, section=None, default=None):
    """
    Fonction générique pour extraire une valeur à partir d'une clé dans un
    fichier texte.
    Args:
        - file_path (str): Chemin du fichier à analyser.
        - key (str): Clé à rechercher (ex: "MISS_DISTANCE").
        - section (str, optional): Section où chercher la clé (ex: "OBJECT2").

    default: Valeur par défaut si la clé n'est pas trouvée.
    Returns: Valeur extraite ou la valeur par défaut si non trouvée.
    """
    try:
        in_target_section = section is None # Si section est None, on
        cherche partout
        with open(file_path, 'r', encoding='utf-8') as file:
```

```
for line in file:
    line = line.strip()

    # Vérifier si on entre dans la section cible
    if section and line.startswith("OBJECT") and section in line:
        in_target_section = True
        # Vérifier si on sort de la section cible

    elif section and in_target_section and (line == "" or
(line.startswith("OBJECT") and section not in line)):
        in_target_section = False
    # Si on est dans la section cible et que la clé est trouvée
    if in_target_section and key in line:
        # Extraire la valeur après le signe '='
        parts = line.split('=', 1)
        if len(parts) > 1:
            value = parts[1].strip()
            return value
    return default
except Exception as e:
    print(f"Erreur lors de l'extraction de {key}: {e}")
    return default
```

Figure 21 Fonction python `extract_value()`

Il a fallu extraire les données ainsi lorsque j'ouvre un fichier je cherche les données en regardant la ligne exemple pour la donnée `MISS_DISTANCE` mais grâce à l'héritage le module `Miss_Distance` peut extraire les données qui l'intéresse.

```
def extract_miss_distance(self, file_path):
    """
    Extrait la distance de rapprochement (MISS_DISTANCE) d'un fichier TXT.
    """
    return self.extract_numeric_value(file_path, "MISS_DISTANCE")
```

Figure 22 Fonction python `extract_miss_distance()`

2.2.2.6.4. Conjonction

Pour une même alerte il peut y avoir plusieurs fichiers, en effet puisqu'un satellite fait le tour de la terre toutes les 1h30 il passe devant le capteur toutes les 1 h 30.

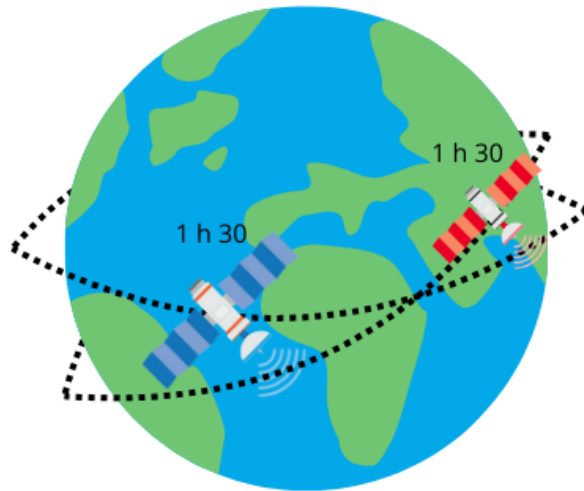


Figure 23 Schéma conjonction

Il a donc été nécessaire de développer un petit algorithme permettant d'afficher, pour une même alerte, toutes ses conjonctions, car dans de nombreux cas, il est essentiel de compter le nombre d'alertes plutôt que le nombre de fichiers. Pour cela, j'ai eu l'idée d'utiliser un dictionnaire dont la clé correspond au nom du satellite extérieur, et la valeur à la liste des fichiers contenant ce satellite comme alerte. Une fois le dictionnaire rempli, il suffit de parcourir chaque section et de comparer les données. Le critère principal de comparaison est la date : si l'écart entre deux conjonctions est inférieur à **1h30**, il s'agit de la même alerte. En revanche, si l'écart est supérieur, il s'agit d'alertes distinctes.

Ainsi j'ai trouvé l'idée de rassembler grâce à un dictionnaire tous les fichiers ayant le même satellite comme alerte. Ainsi j'ai obtenu pour chacun une liste de fichier, que j'ai ensuite ouvert afin d'extraire les données des dates et j'ai vérifié s'il s'agissait de conjonction avec cette méthode.

```
@staticmethod
def is_conjunction(date1: str, date2: str) -> bool:
    try:
        date_format = "%Y-%m-%dT%H:%M:%S.%f"
        dt1 = datetime.strptime(date1, date_format)
        dt2 = datetime.strptime(date2, date_format)

        deltaT = abs((dt1 - dt2).total_seconds())
        return deltaT <= 5400 # 1 h 30 en secondes
    except ValueError as e:
        print(f"Erreur lors de l'analyse des dates : {date1} ou {date2}.
        Erreur : {str(e)}")
        return False
```

Figure 24 Fonction python is_conjunction()

2.2.2.6.5. Distribution par pays

La distribution des pays présente une difficulté, car de nombreux satellites n'appartiennent pas directement à un pays, mais à des organisations privées (comme **SpaceX**). Pour résoudre ce problème, j'ai proposé d'utiliser un fichier **Excel** pour stocker les correspondances entre les organisations et les pays. Lors de l'extraction des données, le programme compare l'organisation du satellite avec les informations contenues dans ce fichier, afin de récupérer automatiquement le pays associé. Et s'il rencontre un nouvel opérateur il demande à l'utilisateur de renseigner le pays de l'opérateur et il sera automatiquement inséré dans le fichier *country_2025.xlsx*.

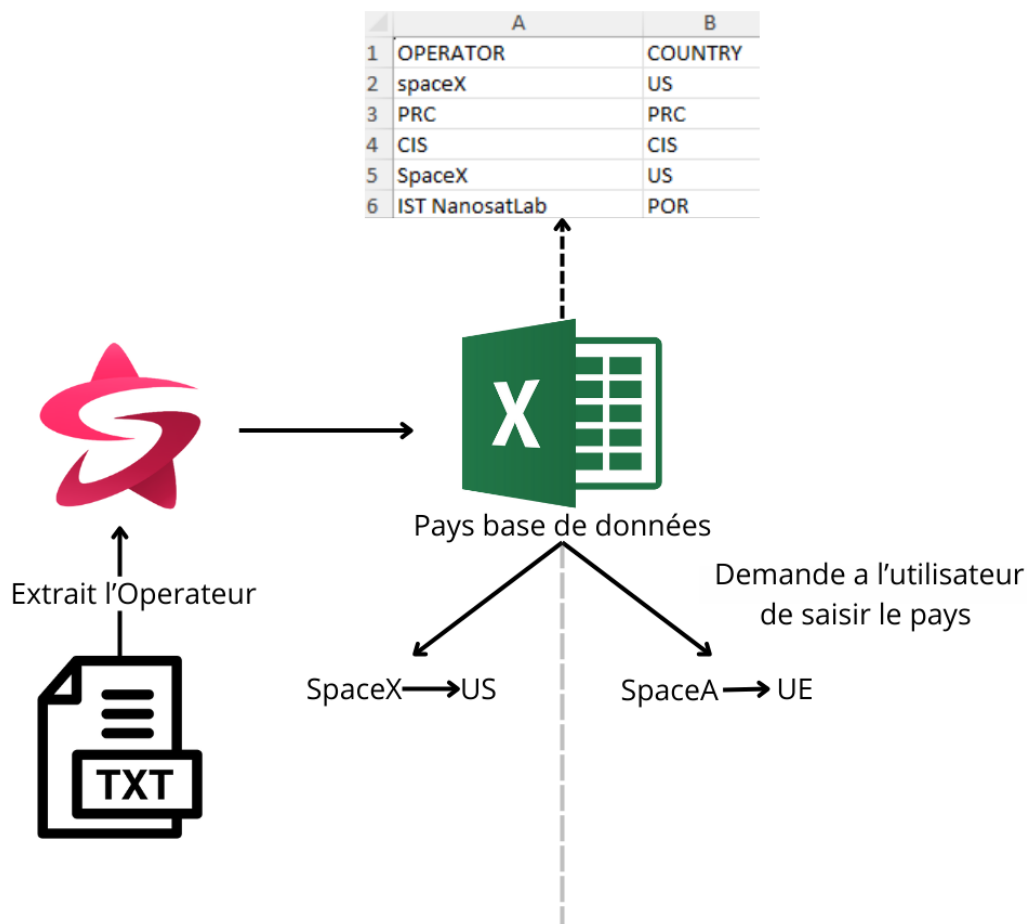


Figure 25 Schéma du fonctionnement des pays

```
def get_operator_country_mapping(database_path =
os.path.join(os.path.dirname(os.path.abspath(__file__)),
"../../config/Country_2025-01-24.xlsx")):
    try:
        workbook = load_workbook(database_path)
        sheet = workbook["OPERATOR"]
        # Trouver l'index de la colonne "COUNTRY"
        country_col = None
        for cell in sheet[1]:
```

```

if cell.value == "COUNTRY":
    country_col = cell.column_letter
    break
# Créer le mapping
mapping = {}
for row in sheet.iter_rows(min_row=2):
    operator = row[0].value
    if operator and sheet[f"{country_col}{row[0].row}"].value:
        mapping[operator] = sheet[f"{country_col}{row[0].row}"].value
return mapping

```

Figure 26 Fonction python get_operator_country_mapping()

2.2.2.7. Demande de déclaration de logiciel

Il m'a également été proposé de soumettre une demande de déclaration de logiciel via le formulaire du CSUM afin d'obtenir un brevet propriétaire pour le logiciel Star Guardian.

2.2.3. Validation

2.2.3.1. Création d'un .exe

Après avoir fait un premier retour à la personne qui m'avait fait la commande du logiciel je me suis rendu compte qu'il était vraiment complexe pour une personne non initiée au programme d'utiliser python et les commandes dans un terminal. Ainsi j'ai réalisé une interface graphique pour répondre à cette problématique. Un fois fait j'ai utilisé la commande python « Pyinstaller » qui permet de transformer un projet python en exécutable avec au minimum un windows 10 32 bits. Qui peut être maintenant transféré et exécuté sur n'importe quelle machine Windows.

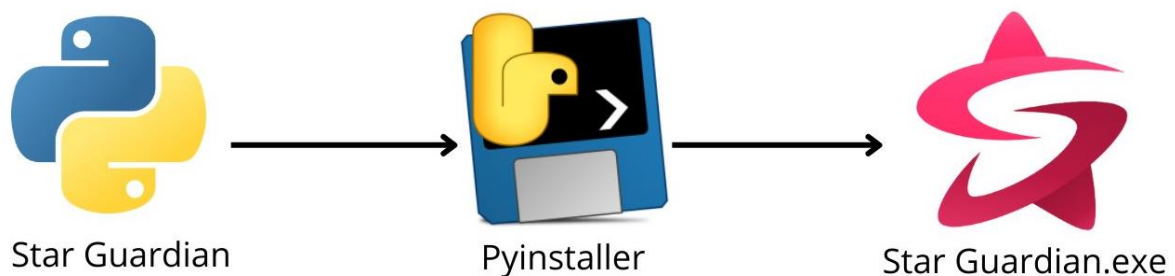


Figure 27 Schéma utilisation de pyinstaller

 StarGuardian.exe	12/03/2025 11:47	Application	47 572 Ko
--	------------------	-------------	-----------

Figure 28 Capture d'écran StarGuardian.exe

Commande utilisé : pyinstaller --add-data
"C:\Users\amich\Documents\Star_Gardian_Project\StarGuardianProject\config;config" --add-
data "C:\Users\amich\Documents\Star_Gardian_Project\StarGuardianProject\gui;gui" --onefile --
icon="C:\Users\amich\Documents\Star_Gardian_Project\icon.ico" --windowed --name
"StarGuardian" main.py

2.2.4. Difficultés rencontrées

2.2.4.1. Conjonction

L'algorithme pour trouver les conjonctions a été complexe à être réalisé car le calcul des dates était trop disparate. J'ai donc cherché une solution pour optimiser et rendre fonctionnelle cette pratique.

1.2.1.1.3. Création du .exe

La création du fichier .exe a été longue et laborieuse. En effet, la bibliothèque « pyinstaller » ne fonctionnait pas en raison d'une mauvaise gestion des packages avec Python. J'ai donc décidé de résoudre le problème à la source en procédant à une installation de Python dans un environnement virtuel (venv), ce qui a permis de régler la situation.

2.2.4.2. Pays

Il a fallu stocker les pays et les traiter mais il y a un problème, en effet les satellites ont un opérateur qui n'est pas toujours une instance étatique (USA, Europe, Chine, etc) et peuvent être parfois une agence privée comme SpaceX, ou même par exemple dans le cas de l'université de Montpellier. Pour résoudre ce problème j'ai donc créé une base de données qui permet de stocker et de lier un opérateur à un pays comme par exemple l'Université de Montpellier à la France. Ainsi à chaque fois que le programme trouve un nouvel opérateur qui n'a pas de pays, il propose à l'utilisateur de le rajouter pour la prochaine fois.

2.3. Apprentissages Critiques mobilisés

- **AC22.02** : Utiliser des techniques algorithmiques adaptées pour des problèmes complexes (par ex. recherche opérationnelle, méthodes arborescentes, optimisation globale, intelligence artificielle...)
- **AC32.03** : Choisir et utiliser des bibliothèques et méthodes dédiées au domaine d'application (imagerie, immersion, intelligence artificielle, jeux vidéo, parallélisme, calcul formel...)
- **AC23.01** : Concevoir et développer des applications communicantes

Rapport de Stage au CSUM

- **AC34.03** : Appliquer des méthodes d'exploration et d'exploitation des données (apprentissage, informatique décisionnelle ou fouille de données)

3. Refactorisation du code de la MCC

Le CSUM dans le cadre de ses activités doit communiquer avec ses satellites en orbite autour de la terre, ainsi pour communiquer le centre spatial a développé un programme informatique permettant de communiqué avec le satellite celui-ci s'appelle le MCC ou Mission Control Center. La liaison avec le satellite se fait au travers de différentes couches protocolaire et réseaux.

3.1. Cahier des Charges

3.1.1. Besoins fonctionnels

- Optimiser et simplifier l'interface entre la MCC et les encodeurs/décodeurs afin de faciliter l'intégration et l'interopérabilité.
- Améliorer la clarté et la structuration du code pour le rendre plus lisible, compréhensible et facile à maintenir.

3.1.2. Besoins non fonctionnels

- **Performance :**
 - La performance ne doit pas baisser ou même doit être plus rapide.
- **Fiabilité :**
 - Les données decoder / encoder doivent être juste pour ne pas poser de problème de transcription.

3.2. Rapport technique

3.2.1. Conception

3.2.1.1. Architecture générale

Le processus de communication fonctionne grâce à une parabole ou antenne qui communique avec un satellite grâce à une encapsulation de protocoles de communication pour transmettre les données. Ainsi, il existe 2 chemins l'un pour l'encodage donc l'Uplink (Envoie) et le second pour le DownLink (téléchargement). Le problème actuel réside dans la conception des codecs en effet, il existe de nombreux codec, connecté à la MCC, chacun de ces codecs est connecté individuellement à la MCC, ce qui alourdie l'interface entre les codecs et la MCC et complexifie la maintenance. Les encodeurs reçoivent des paquets formés par des octets, encapsule les paquets dans un autre protocole et retournent un bitstream. Et les décodeurs font l'opération inverse. Ils reçoivent un bitstream de GNURadio, réforme des paquets, et désencapsule ces paquets.

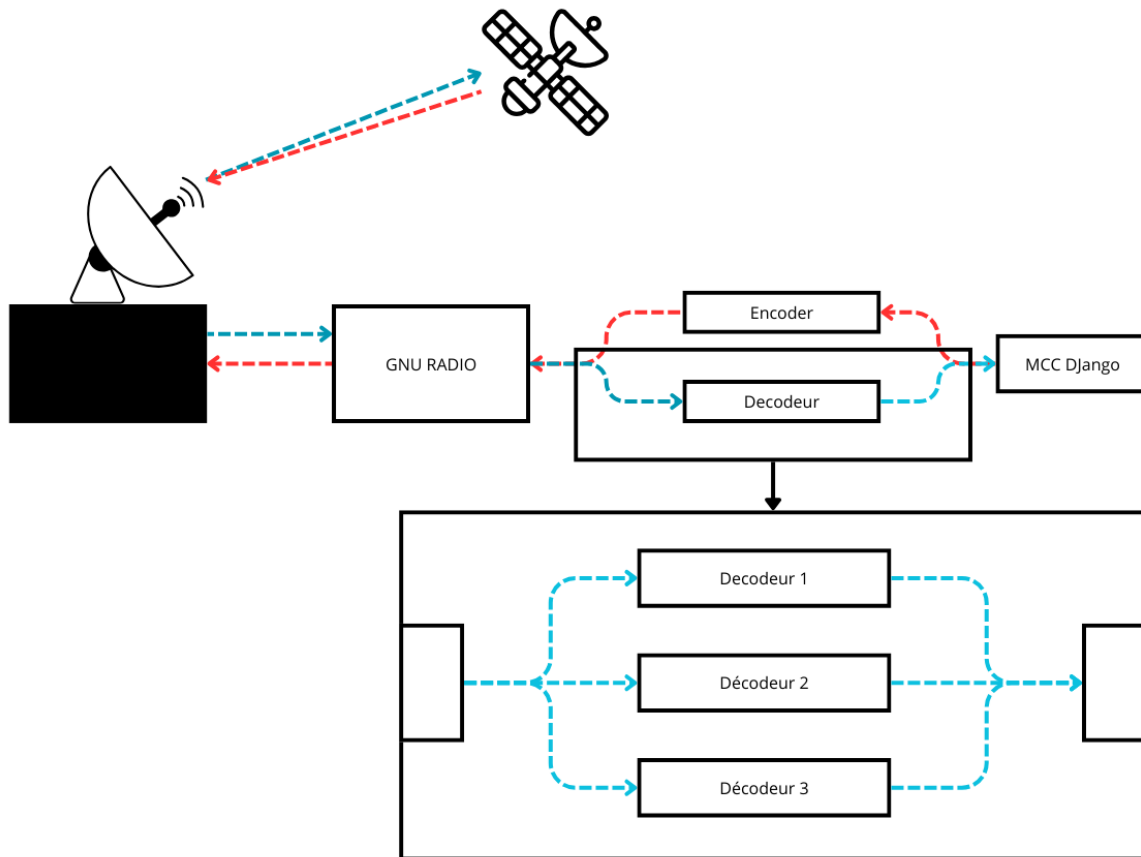


Figure 29 Schéma de la communication GS / Satellites

Les encodeurs/décodeurs se trouvent entre les modem et l'interface MCC, ils reçoivent en entrée des paquets sous forme de bytes et renvoie dans le cadre du décodeur les données extraites et dans le cadre de l'encodeur les données encodées suivant le protocole.

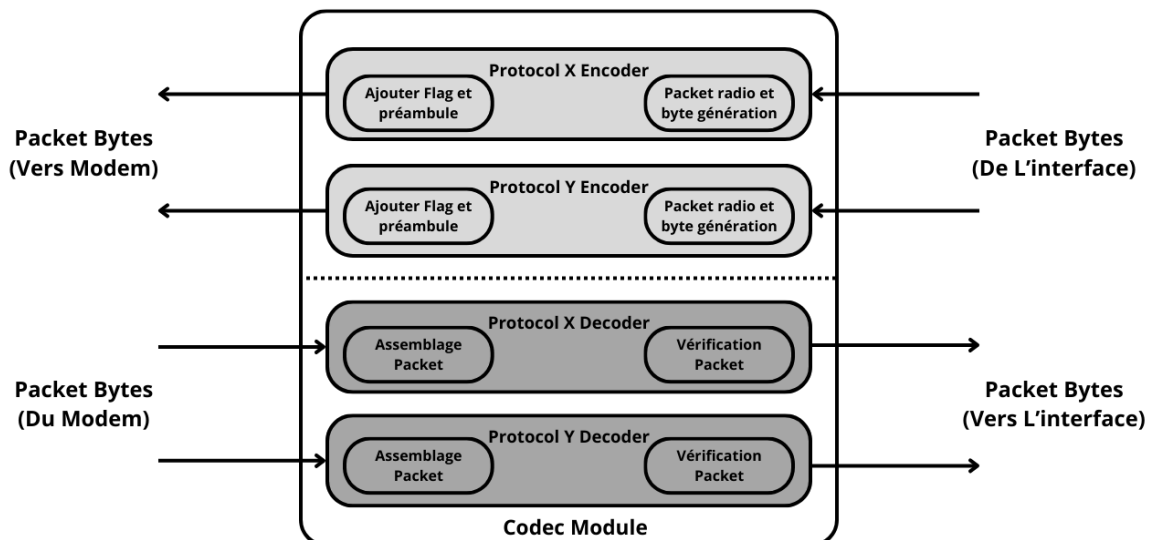


Figure 30 Schéma protocole encodeur / décodeur

3.2.2. Réalisation

3.2.2.1. Héritage et restructuration

Pour commencer, j'ai mis en place un mécanisme d'héritage afin de refactorer la logique des codecs. À l'origine, il y avait beaucoup de duplication de code. J'ai donc entrepris de centraliser cette duplication en la regroupant dans une classe par héritage. Cependant, comme les routeurs ne peuvent fonctionner que dans un seul sens, j'ai créé un second héritage pour que le routeur encodeur ne gère que les encodeurs et que le routeur décodeur ne gère que les décodeurs. Ainsi, cela m'a permis de créer des encodeurs et décodeurs concrets qui seront utilisés pour encoder et décoder les données.

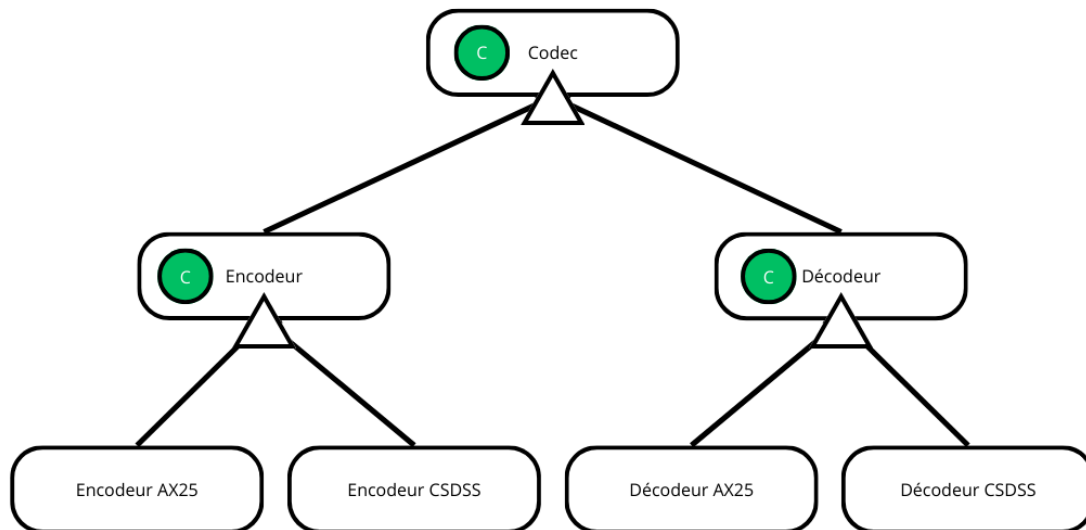


Figure 31 Schéma héritage des codecs

3.2.2.2. Routeur interface

Grace a la refactorisation et au principe d'héritage, j'ai pu créer un routeur qui prend en entrée des données et qui grâce à l'identification du protocole va rediriger le flux vers le bon décodeur ou encodeur.

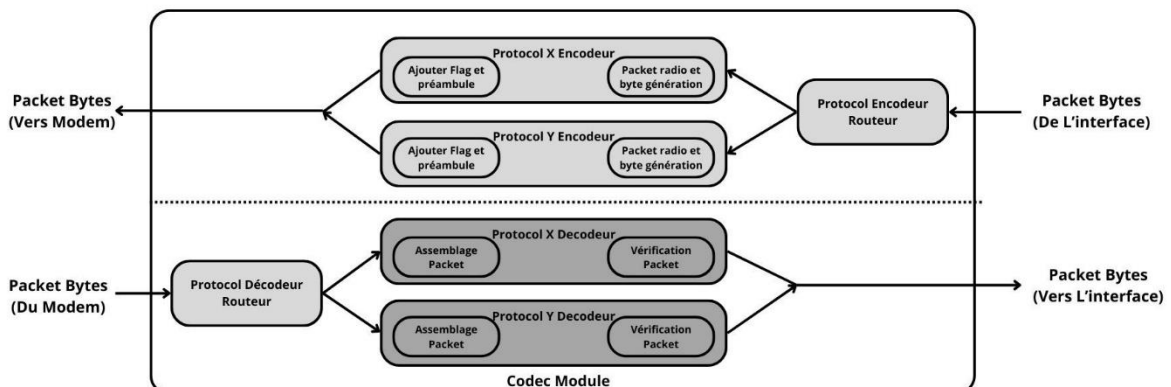
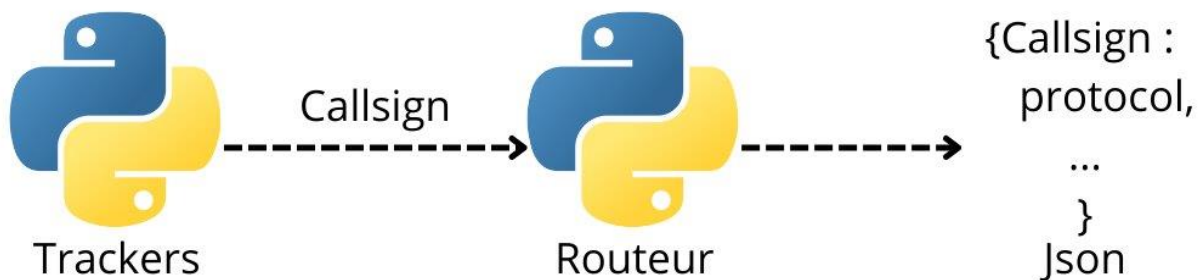


Figure 32 Modélisation de la solution avec Routeur

3.2.2.3. Identification des satellites

Pour faire fonctionner le routage, il me fallait identifier le satellite avec son protocole afin de lui faire emprunter la bonne voie. Heureusement, chaque satellite possède un « Callsign » littéralement indicatif d'appel qui permet d'identifier une station sols, navire, avion ou satellites. Ainsi, j'ai créé un Json avec comme nom le Callsign et le protocole désirés.

Cependant, le CSUM utilise 2 systèmes pour communiquer avec ses satellites la bande S* ou l'UHF* qui n'utilise pas les mêmes fonctionnements. L'UHF utilise l'AX.25 et la bande S utilise le CCSDS. J'ai donc amélioré le fonctionnement en créant un Json avec 2 champs pour un même Callsign. Le premier reprend toutes les datas que l'on veut transmettre et le second prend une liste de métadonnées comme l'adresse de la station sols.



Détection des codecs

Pour faciliter le code de cette solution, il m'a été conseillé de réaliser une détection de fichiers codec. Chaque décodeur/encodeur étant un fichier python ça a rendu le travail plus simple ainsi, j'ai écrit un détecteur de codec qui va analyser le dossier codecs en temps réel afin de voir quel codec s'y trouve. Ainsi, si un codec est ajouté ou retiré, il le détectera.

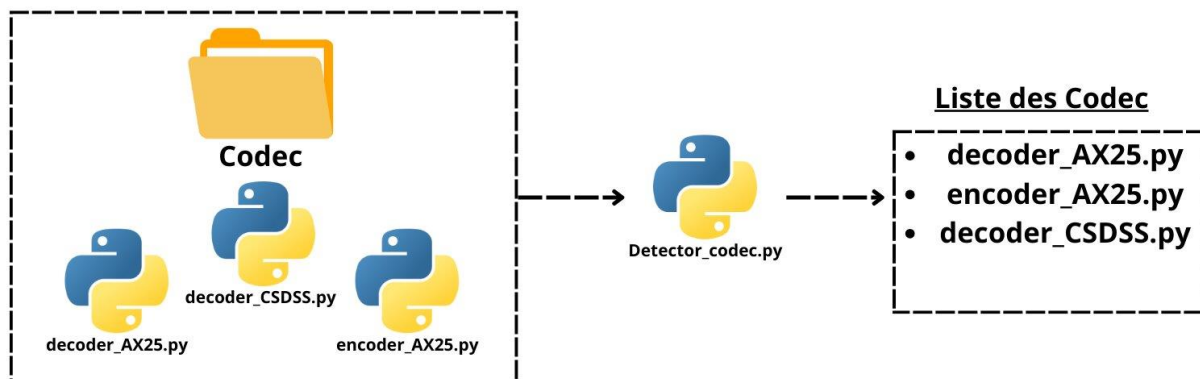


Figure 33 Schéma détecteur de codec

3.2.3. Validation

3.2.3.1. C

A Compléter

3.2.4. Difficultés rencontrées

3.2.4.1. Setup des environnements

Le code de l'MCC fonctionne avec de très nombreuses librairies et dépendance qui sont installés dans un environnement virtuel python, le problème étant que l'MCC a été codé sur python 3.11. Et que ces librairies ont continué d'évoluer ainsi lorsque les commandes "pip install" la commande pour installer les librairies a été exécuter plusieurs librairies n'arriver pas à s'installer, car elle avait changé de nom entre temps. Comme par exemple la librairie crc qui avait évolué en crccheck ce qui rendait les imports caducs.

3.2.4.2. Compréhension d'un code étranger

Le code de l'MCC a été codé il y a plusieurs années et a été petit à petit mis à jour avec de nouvelles fonctionnalités avec un développement itératif non suivis et documenté ce qui a rendu le code très brouillons et complexe à comprendre, ainsi mon plus grand temps a été d'essayer de comprendre comment fonctionner le programme et de comprendre sa logique qui n'était pas toujours la mienne.

3.3. Apprentissages critiques mobilisés

AC31.03 : Intégrer des solutions dans un environnement de production

AC22.02 : Utiliser des techniques algorithmiques adaptées pour des problèmes complexes (par ex. recherche opérationnelle, méthodes arborescentes, optimisation globale, intelligence artificielle...)

AC23.01 : Concevoir et développer des applications communicantes

AC36.01 : Organiser et partager une veille technologique et informationnelle

4. Rapport d'activité pour l'ensemble du stage.

4.1. Installation de postes pour le personnel

Au cours de mon stage, j'ai participé activement au déploiement et à la maintenance des postes informatiques destinés au personnel du centre. Cette mission comprenait le démontage des anciens équipements, l'installation de nouveaux postes et leur configuration selon les besoins des utilisateurs.

J'ai ainsi procédé à l'installation et à la configuration de plusieurs systèmes d'exploitation, notamment **Windows 10 et 11**, ainsi que des distributions Linux comme **Ubuntu 22.04 et 24.04 ou Debian 10, 11 et 12**. Chaque installation a été suivie d'une phase de mise à jour, d'installation des logiciels nécessaires et de vérification du bon fonctionnement des machines.

Cette expérience m'a permis de renforcer mes compétences en administration système, en gestion de parc informatique et en support technique auprès des utilisateurs.

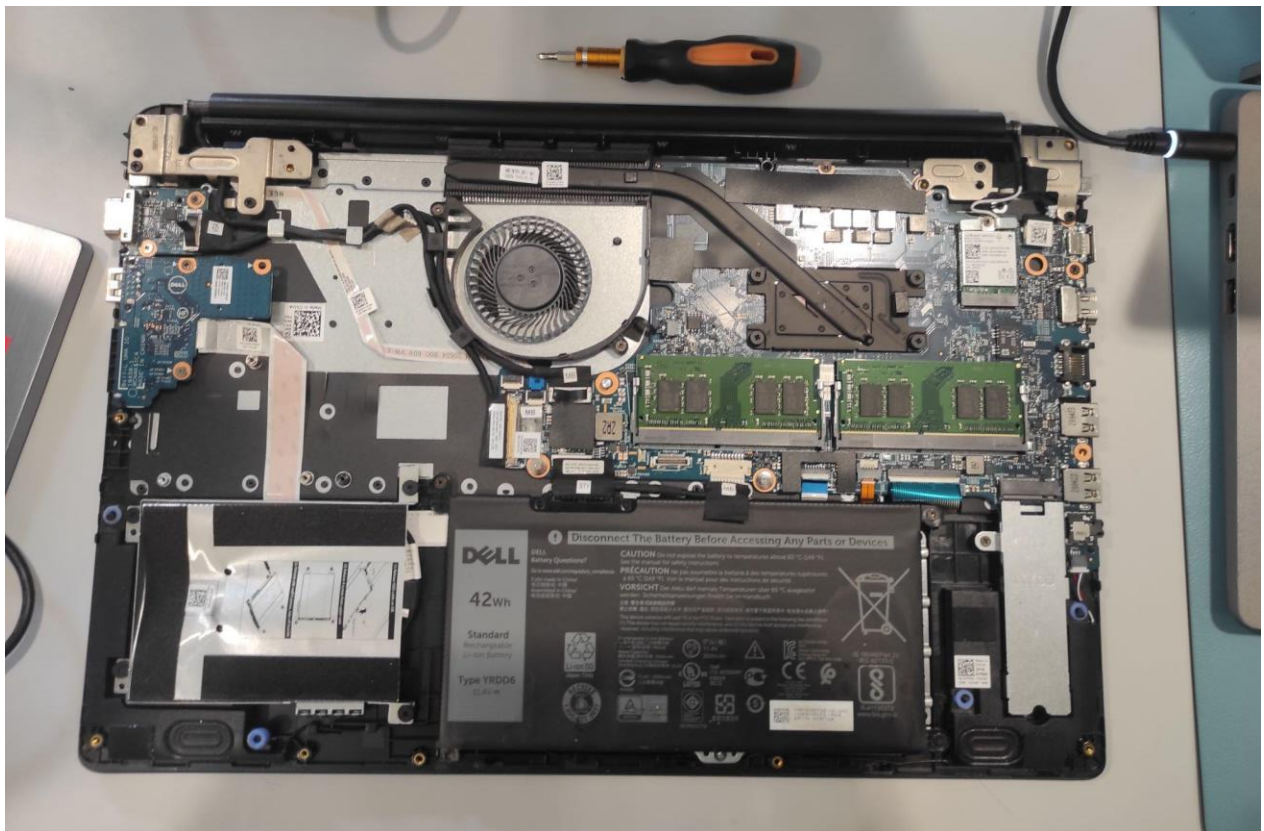


Figure 34 Photo ordinateur démonté

5. Gestion de projet

Lors de ma première mission, j'ai rapidement adopté une organisation efficace. Après chaque test et recherche de solution, j'échangeais avec mon tuteur de stage à travers des débriefings réguliers. Ces discussions permettaient d'affiner les demandes et d'éviter de s'éloigner des objectifs initiaux. Cette méthode de travail a également été appliquée aux deux autres projets auxquels j'ai été confronté. À chaque avancée de mes réalisations, je communiquais avec les parties prenantes pour assurer une bonne adéquation avec leurs attentes.

- **Conception**

Avant d'entamer toute implémentation, j'analysais la problématique en profondeur. J'utilisais des diagrammes, des schémas et des réflexions approfondies pour structurer ma démarche et identifier la meilleure approche à adopter.

- **Mise en œuvre**

Une fois la solution définie, je procédais au développement ou à l'implémentation des corrections nécessaires. Durant cette phase, il était fréquent que de nouveaux défis apparaissent. Dans ce cas, je réappliquais l'ensemble de la méthodologie pour les résoudre de manière efficace.

- **Tests et validation**

Après l'implémentation, je m'assurais du bon fonctionnement de la solution en réalisant des tests rigoureux, notamment des tests unitaires. Mon objectif était de détecter d'éventuelles régressions ou anomalies avant la mise en production.

- **Optimisation et documentation**

Une fois les tests validés, j'optimisais le code et l'environnement en effectuant un nettoyage : suppression des éléments inutiles, amélioration de la lisibilité, et documentation des fonctionnalités pour faciliter leur maintenance future.

- **Versioning et sauvegarde**

Enfin, chaque évolution était systématiquement enregistrée sur les dépôts via des commits bien documentés, garantissant ainsi un suivi précis des modifications et une meilleure collaboration avec les autres membres de l'équipe.

Conclusion

Bilan du travail réalisé

La solution que j'ai trouvée a permis de faciliter et d'accélérer le clonage des postes stagiaires. Cette méthode a permis de faciliter et d'accélérer la vitesse d'installation des postes passant de 40 min par postes à 40 pour 4 postes en simultané.

Puis j'ai réalisé un logiciel permettant d'analyser et transférer des données vers un fichier excel, ce logiciel permet ainsi de faciliter grandement l'étude des données des alertes collision car tout est fait automatiquement contrairement à avant où tout devait être fait à la main avec une suite de copier / coller.

J'ai également travaillé sur l'architecture de codeurs / décodeurs de protocoles. Cela a permis de rendre la compréhension et l'utilisation de ces encodeurs plus simple et efficace.

J'ai également aidé mon tuteur de stage dans ses activités en installant et configurant des postes pour les membres du CSUM. Lui libérant ainsi un temps précieux, chaque installation prenant des heures.

Perspectives

Le déploiement des postes informatiques pourrait être automatisé, notamment lors de l'installation de nouveaux postes pour différents types d'utilisateurs, comme les Masters.

Le déploiement de postes informatiques peut ainsi être automatisé avec d'autre installation de poste comme un nouveau type de personne comme les Master.

Pour le programme Star Guardian puisque le code est SOLID il est facile de rajouter des analyses sur de nouvelles données qui peuvent être traité.

Le travail que j'ai effectué sur la MCC pourra être utilisé dans les prochaines versions de cette MCC.

Bilan des acquis techniques

J'ai acquis des compétences en administration d'un réseau local, notamment en configuration et gestion des connexions. J'ai également appris à déployer un système d'exploitation personnalisé sur un réseau, optimisant ainsi l'installation et la maintenance des machines.

En développement, j'ai réalisé une application en Python en intégrant des bibliothèques que je ne connaissais pas auparavant, comme Matplotlib et Pandas. Cela m'a permis de mieux comprendre leur fonctionnement et leur utilisation.

Enfin, j'ai travaillé sur la refactorisation de code en appliquant les principes SOLID, améliorant ainsi la lisibilité, la modularité et la maintenabilité du programme.

- **Développement d'une application en Python**
- **Utilisation de bibliothèques Python inconnues (Matplotlib, Pandas)**
- **Utilisation et compréhension des applications tournant sous Django**
- **Administration d'un réseau local**
- **Déploiement d'un OS personnalisé sur un réseau**
- **Refactorisation de code en appliquant les principes SOLID**

Bilan de l'expérience en entreprise

Cette immersion dans le monde du travail représente pour moi le pas idéal. Ce stage m'a appris beaucoup sur le fonctionnement du métier d'informaticien dans le milieu professionnel. J'ai pu voir différentes facettes de plusieurs métiers de l'informatique, du développeur à l'administrateur système et réseau.

J'ai pu aborder plusieurs concepts nouveaux et caractéristiques du monde du travail.

- Collaborer avec des personnes non familiarisées avec l'informatique.
- Comprendre et assimiler les demandes d'un administrateur système et réseau.
- S'entourer de collaborateur technique afin de partager des idées et connaissances dans le but de faire progresser l'outil.

Bilan de l'expérience personnelle

J'ai vécu une expérience très enrichissante tant sur le plan technique qu'humain. J'ai appris de nombreuses choses que je n'avais pas étudiées en cours, et j'ai eu l'occasion d'approfondir mes connaissances en infrastructure réseau avec des concepts nouveaux qui me seront toujours utiles.

J'ai énormément apprécié l'accueil qui m'a été réservé, et j'ai su tisser des liens agréables avec l'ensemble des membres du centre. Je me suis particulièrement senti fier et valorisé par la grande marque de confiance que le centre m'a accordée en me confiant de travailler sur l'évolution de l'architecture de la Mission Control Center, ce qui m'a permis de mettre en œuvre à la fois ma passion pour le développement et les outils appris lors de mon cursus universitaire.

Bibliographie

- [1]
« 2. Traitement de données avec PANDAS — Outils numériques avancés en Mécanique ». Consulté le: 14 mars 2025. [En ligne]. Disponible sur: https://perso.univ-lyon1.fr/marc.buffat/COURS/BOOK_OUTILSNUM_HTML/MGC2367M/cours/Chap3_PandasBigdata.html
- [2]
Graven - Développement, *APPRENDRE LE PYTHON #9 ? INTERFACE GRAPHIQUE (avec Tkinter)*, (18 février 2019). Consulté le: 14 mars 2025. [En ligne Vidéo]. Disponible sur: <https://www.youtube.com/watch?v=N4M4W7JPOL4>
- [3]
malekalmorte, « Clonezilla et DRBL : cloner des PC par le réseau », malekal.com. Consulté le: 14 mars 2025. [En ligne]. Disponible sur: <https://www.malekal.com/clonezilla-drbl-cloner-reseau/>
- [4]
Heikki Koivisto, *Comment créer une image de clone Windows 10 avec le serveur FOG 2019*, (29 octobre 2019). Consulté le: 14 mars 2025. [En ligne Vidéo]. Disponible sur: <https://www.youtube.com/watch?v=rQtYX9FynLg>
- [5]
Chrtophe, « Déployer vos applications avec Fogproject », Developpez.com. Consulté le: 14 mars 2025. [En ligne]. Disponible sur: <https://chrtophe.developpez.com/tutoriels/deploiement-fogproject/>
- [6]
« Diskless Remote Boot in Linux (DRBL) », SourceForge. Consulté le: 14 mars 2025. [En ligne]. Disponible sur: <https://sourceforge.net/projects/drbl/>
- [7]
« DRBL - About ». Consulté le: 14 mars 2025. [En ligne]. Disponible sur: <https://drbl.org/>
- [8]
« FOG Project ». Consulté le: 14 mars 2025. [En ligne]. Disponible sur: <https://fogproject.org/>
- [9]
« GNU/Linux | Mettre en place un serveur de déploiement Clonezilla », STD.ROCKS. Consulté le: 14 mars 2025. [En ligne]. Disponible sur: https://std.rocks/fr/gnulinux_clonezilla_drbl.html
- [10]
« Interface graphique Tkinter python ». Consulté le: 14 mars 2025. [En ligne]. Disponible sur: <https://python.doctor/page-tkinter-interface-graphique-python-tutoriel>
- [11]
« Introduction à Pandas ». Consulté le: 14 mars 2025. [En ligne]. Disponible sur: <https://www.python-simple.com/python-pandas/panda-intro.php>
- [12]
« Pandas », *Wikipédia*. 22 août 2024. Consulté le: 14 mars 2025. [En ligne]. Disponible sur: <https://fr.wikipedia.org/w/index.php?title=Pandas&oldid=217899359>
- [13]
« pandas (software) », *Wikipedia*. 20 février 2025. Consulté le: 14 mars 2025. [En ligne]. Disponible sur: [https://en.wikipedia.org/w/index.php?title=Pandas_\(software\)&oldid=1276818895](https://en.wikipedia.org/w/index.php?title=Pandas_(software)&oldid=1276818895)
- [14]
« PyInstaller Manual — PyInstaller 6.12.0 documentation ». Consulté le: 14 mars 2025. [En ligne]. Disponible sur: <https://pyinstaller.org/en/stable/>

[15]

« Tkinter », *Wikipédia*. 12 mai 2024. Consulté le: 14 mars 2025. [En ligne]. Disponible sur: <https://fr.wikipedia.org/w/index.php?title=Tkinter&oldid=215033040>

[16]

« tkinter --- Python interface to Tcl/Tk », Python documentation. Consulté le: 14 mars 2025. [En ligne]. Disponible sur: <https://docs.python.org/3/library/tkinter.html>

[17]

« utilisateurs:ool:czillaserv [Wiki ubuntu-fr] », Consulté le: 14 mars 2025. [En ligne]. Disponible sur: <https://doc.ubuntu-fr.org/utilisateurs/ool/czillaserv>

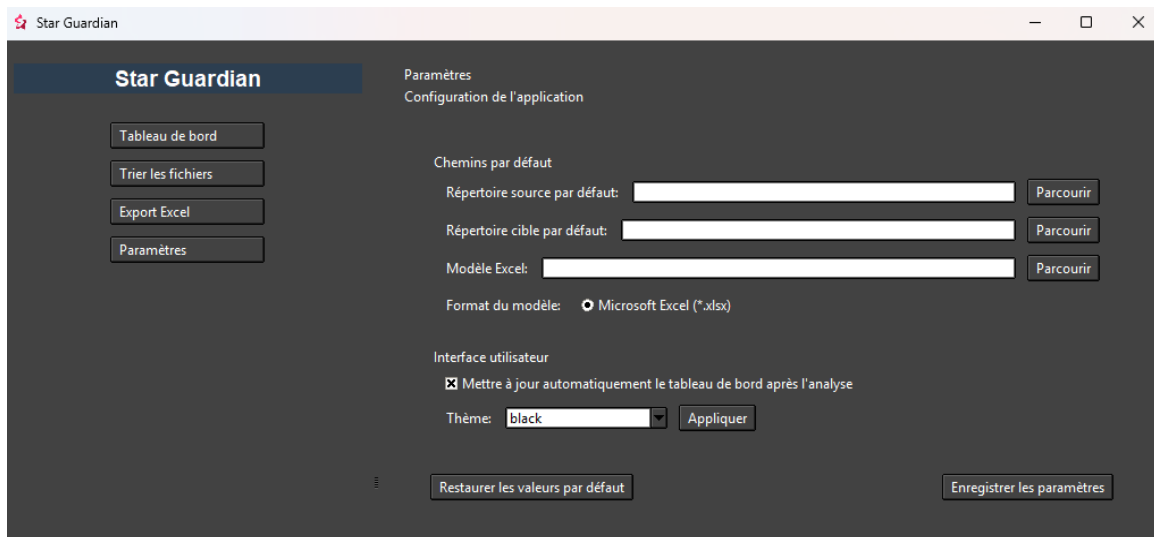
[18]

« WinCDEmu - the easiest way to mount an ISO. And more... » Consulté le: 14 mars 2025. [En ligne]. Disponible sur: <https://wincdemu.sysprogs.org/>

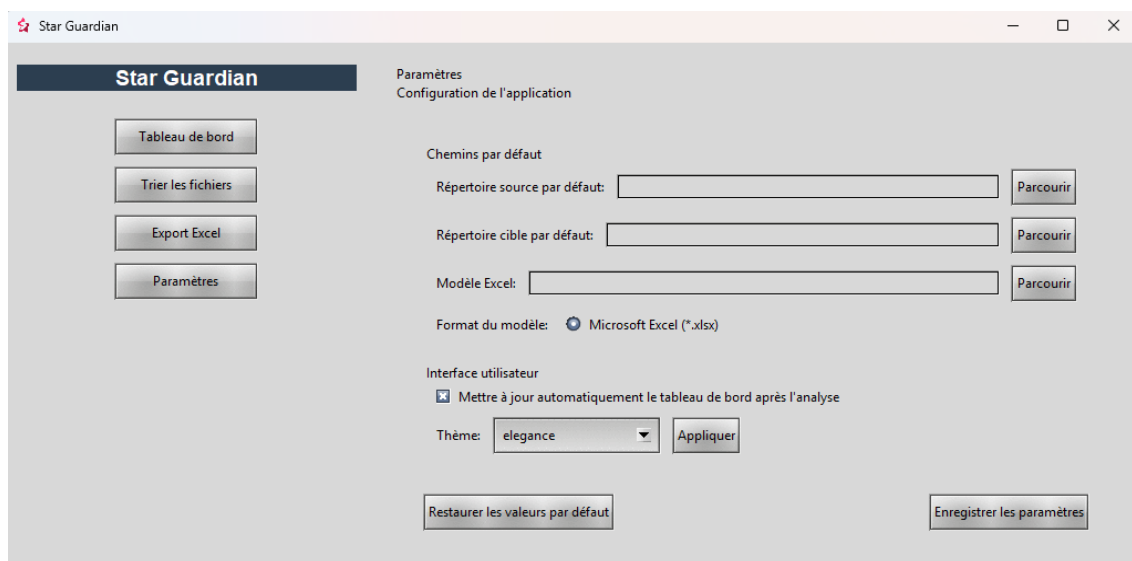
Annexes

Annexe 1 Capture d'écran paramètres theme Dark	54
Annexe 2 Capture d'écran paramètres thème Elégance	54
Annexe 3 UHF du CSUM	54
Annexe 4 Bande S du CSUM	54
Annexe 5 Diagramme de classe complet du backend de Star Guardian	55
Annexe 6 Organigramme RH du CSUM	56

Rapport de Stage au CSUM



Annexe 1 Capture d'écran paramètres thème Dark



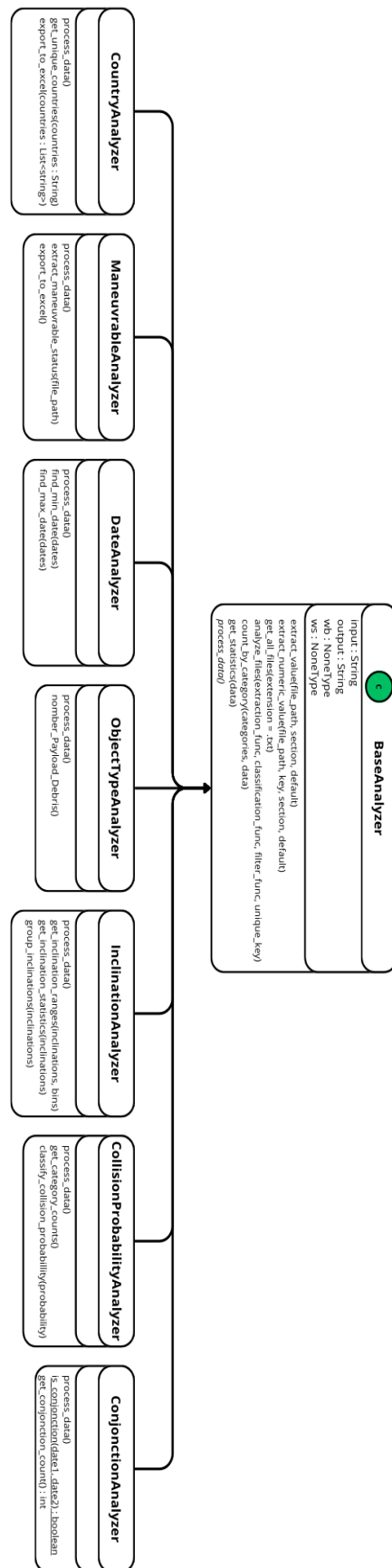
Annexe 2 Capture d'écran paramètres thème Elégance



Annexe 3 UHF du CSUM

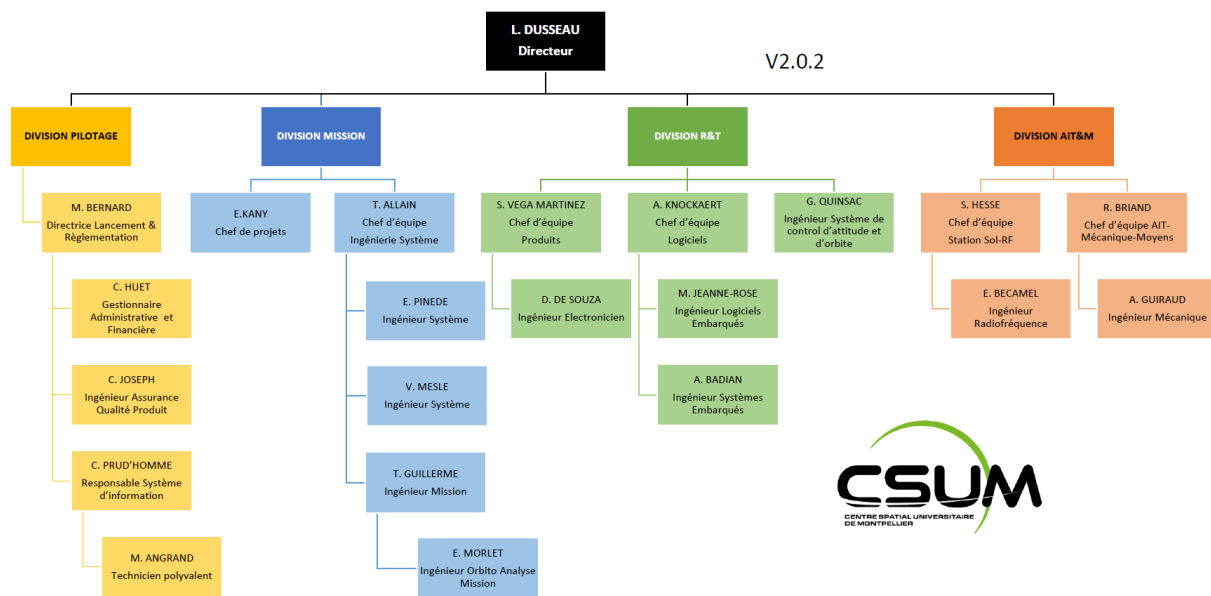


Annexe 4 Bande S du CSUM



Annexe 5 Diagramme de classe complet du backend de Star Guardian

Rapport de Stage au CSUM



Annexe 6 Organigramme RH du CSUM